SAI VIDYA INSTITUTE OF TECHNOLOGY

(AFFILIATED TO VTU)

PROJECT DOCUMENTATION ON

Automated Resume Analyzer with Job Fit Scoring System Using Apache Tika

SUBMITTED BY

Deeksha M (CSE) – 1VA22IS039

Hithashree R (ISE) – 1VA22IS037

Lavanya M R (CSE) – 1VA22CS060

Rakshitha N (CSE) – 1VA22CS097

INDEX

ABSTRACT:

Resume Analyzer Pro is a web-based application developed to simplify the process of resume screening, keyword matching, and role recommendation for job seekers and administrators. The system allows a user to upload a resume file or paste resume text, compare it against a target job description, and obtain a structured analysis that reflects keyword coverage, tracked skill match, and document completeness. The project addresses a practical problem faced by candidates and recruiters: resumes are often rejected early because relevant skills are missing, under-emphasized, or presented in a way that does not align with job descriptions.

The application is implemented with Spring Boot as the backend framework, Thymeleaf for server-side rendering of web pages, Spring Data JPA for persistence, MySQL for storage, Apache Tika for file content extraction, and iText for downloadable PDF report generation. The project also includes a dedicated administrative module that lets administrators monitor registered users, inspect saved analyses, review uploaded resume content, and control user access through lock and delete operations.

From a functional point of view, the system evaluates job fit by computing keyword match score, tracked skills score, and resume completeness score, then derives a final score and a user-friendly insight message. It also identifies matched skills, missing skills, and suggested job roles. The project therefore combines usability, data persistence, document parsing, analytics visualization, and report export into a single integrated platform designed for career guidance and resume improvement.

INTRODUCTION:

In present-day recruitment workflows, resumes are frequently processed through Applicant Tracking Systems before they ever reach a human reviewer. These systems usually favor resumes that include the right technical skills, role-specific terminology, and a clear structure. As a result, many promising candidates fail to move forward because their resumes are not aligned with the expected vocabulary of the job description. This creates a need for an accessible and explainable system that helps users understand how their resumes are interpreted.

Resume Analyzer Pro was created to meet this need by giving users a practical interface to upload or paste resume data, submit a job description, and immediately receive a breakdown of resume relevance. The project is not intended to replace recruiters or guarantee hiring outcomes. Instead, it functions as a decision-support and self-improvement tool that highlights strengths, detects gaps, and promotes better resume tailoring for a chosen role.

The system also recognizes that resume analysis should not end with a single score. Candidates need visibility into why a score was generated, which skills were matched, which skills were missing, and what kinds of roles appear most consistent with the job description. For that reason, the application stores analysis history, lets users revisit previous runs, visualizes score progression, and exports analysis reports in PDF format for later reference.

Alongside the candidate-facing features, the project introduces an administration layer that supports platform management. This layer lets an administrator view all users, see how often the platform is used, review analyses across the system, inspect resume details for specific users, and suspend or delete accounts when required. This dual-view design makes the project suitable not only as a student software engineering exercise but also as a realistic prototype of a multi-user resume analysis platform.

OBJECTIVE OF THE PROJECT:

The main objective of Resume Analyzer Pro is to design and implement a reliable software platform that evaluates resume quality against a job description using transparent and understandable scoring logic. The system focuses on making resume feedback immediate, structured, and actionable.

1. Primary Objective

To build a full-stack web application that accepts resume input, parses or stores the content, compares it with a target job description, and produces an analysis report containing overall fit score, score breakdown, matched skills, missing skills, and recommended job roles.

2. Secondary Objectives

To Analyze Resume Against Job Description

One of the major objectives of this project is to evaluate how well a resume matches a given job description by analyzing:

Keywords present in the job description

Relevant skills required for the job

Matching content in the resume

Overall similarity score

This helps in determining the suitability of a candidate for a specific job role.

3. To Identify Matched and Missing Skills

The system identifies skills present in both resume and job description and also highlights missing skills.

Examples include:

Matched skills (e.g., Java, SQL, Spring Boot)

Missing skills (e.g., Hibernate, AWS, Docker)

Skill gap analysis

This helps users understand what skills they need to improve.

4. To Provide Job Role Recommendations

The system suggests suitable job roles based on the user's resume and skill set.

Examples include:

Backend Developer

Full Stack Developer

Data Scientist

Python Developer

This helps users explore relevant career opportunities.

5. To Calculate Resume Completeness

The system checks whether important sections are present in the resume such as:

Education

Skills

Experience

Projects

This ensures that the resume is properly structured and complete.

6. To Maintain User Analysis History

The system stores all user activities and analysis results such as:

Uploaded resumes

Previous analysis reports

Scores and feedback

This allows users to track their progress over time.

7. To Provide Admin Monitoring and Control

The system includes an admin module that allows:

Viewing registered users

Monitoring user activities

Suspending and activating accounts

Deleting user accounts

This ensures proper management and security of the system

SCOPE OF THE PROJECT:

SCOPE OF THE PROJECT

1. Introduction to Project Scope:

The scope of the Resume Analyzer System defines the functional boundaries, capabilities, and limitations of the application. It outlines what the system is designed to perform, the features it includes, and the areas that are not covered within the current implementation.

This project is designed to function as an automated resume evaluation and feedback system. It focuses on analyzing resumes against job descriptions using rule-based techniques and generating meaningful insights such as match scores, skill gaps, and recommendations. The system acts as a decision-support tool for users rather than a final authority in recruitment.

2. Functional Scope:

1.Resume Upload and Processing:

The system allows users to upload resumes in PDF format and processes them using Apache Tika to extract textual content.

• Resume file upload (PDF format)

• Text extraction from resume

• Storage of extracted content in database

• Handling multiple resume submissions

2. Job Description Input and Comparison:

The system enables users to input job descriptions and compares them with resume content.

• Manual entry of job description

• Preprocessing of text (cleaning and normalization)

• Comparison between resume and job description

• Identification of relevant keywords

3. Resume Analysis and Scoring:

The core functionality includes analyzing resumes and generating scores based on defined criteria.

• Keyword match percentage calculation

• Skill match evaluation using predefined dataset

• Resume completeness assessment

• Final score generation using combined metrics

4. Skill Gap Identification and Recommendations:

The system identifies missing skills and suggests improvements.

• Detection of matched and missing skills

• Highlighting skill gaps

• Recommendation of relevant job roles

• Feedback for resume improvement

5. Analysis History and Visualization:

The system maintains records of previous analyses and displays results.

• Storage of all analysis results

• Viewing past analysis history

• Display of scores and results

• Graphical representation of performance trends

3. User-Based Scope:

1. Users (Job Seekers)

Users can:

• Register and login into the system

• Upload resumes and provide job descriptions

• View analysis results and scores

• Access history of previous analyses

• Improve resumes based on feedback

2. Administrator (Admin)

Admin can:

• Login using predefined credentials

• View all registered users

• Monitor user activities and analysis data

• Suspend and activate user accounts

• Delete users and their associated data

4. Technical Scope

1. System Architecture

The project includes:

• Layered architecture (Controller, Service, Repository)

• Modular and maintainable design

• Database-driven application

• Separation of frontend and backend logic

2. Technology Stack

The system is built using:

• Spring Boot for backend development

• Thymeleaf for frontend rendering

• MySQL for database management

• Apache Tika for resume parsing

3. Rule-Based Analysis Logic

The system uses:

• Keyword matching techniques

• Skill comparison using predefined dataset

• Completeness evaluation based on sections

• Weighted scoring for final result

No advanced machine learning models are used in the current implementation.

4. Data Storage and Management

The system manages structured data including:

• User details

• Resume content

• Analysis results

• Skills and recommendations

5. Operational Scope

1. System Usage Environment

The system operates as:

• A web-based application accessible through browsers

• A platform for resume evaluation and feedback

• A tool for assisting job seekers in resume improvement

2. Application Domain

The system is applicable to:

• Students seeking internships or jobs

• Freshers preparing resumes

• Individuals applying for technical roles

• Entry-level recruitment scenarios

3. Limitations and Exclusions (Out of Scope)

The following are outside the scope of this project:

• No AI or machine learning-based semantic analysis

• No integration with real-time job portals

• No automatic resume ranking across multiple candidates

• No verification of resume authenticity

• No support for multiple languages

4. Scalability and Future Expansion Scope

Although not part of the current implementation, the system can be extended to include:

• AI-based resume analysis and NLP techniques

• Integration with job portals (LinkedIn, Indeed, etc.)

• Support for multiple file formats (DOCX, TXT)

• Advanced analytics and dashboard visualization

• Personalized career recommendations

5. Summary of Project Scope

In summary, the scope of the Resume Analyzer System includes:

• Resume parsing and text extraction

• Job description comparison

• Keyword and skill-based analysis

• Score generation and feedback

• User and admin management

The system provides a structured and automated approach to resume evaluation while maintaining simplicity, scalability, and user-friendliness.

FEASIBILITY STUDY OF THE PROJECT:

1. Introduction to Feasibility Study

A feasibility study is conducted to determine whether the Resume Analyzer System can be designed, developed, implemented, and operated effectively within the constraints of time, cost, technology, and user requirements.

This study evaluates the project from multiple perspectives to ensure that it is:

• Practically achievable

• Economically viable

• Technically sound

• Operationally acceptable

• Legally and ethically compliant

2. Technical Feasibility

1. Availability of Technology

The project is technically feasible as it uses widely available and well-established technologies, including:

• Spring Boot for backend development

• Thymeleaf for frontend rendering

• MySQL for structured data storage

• Apache Tika for resume parsing

All these technologies are stable, open-source, and supported by strong developer communities.

2. System Complexity

The system follows a modular architecture consisting of controller, service, and repository layers, which:

• Reduces development complexity

• Allows easy debugging and maintenance

• Supports independent module development

The analysis logic is rule-based, avoiding the need for complex machine learning models.

3. Integration Feasibility

The Resume Analyzer System can be integrated with:

• Job portals for fetching job descriptions

• Resume submission platforms

• Recruitment systems

The use of standard web technologies and APIs ensures compatibility with future integrations.

4. Performance and Scalability

The system supports:

• Efficient processing of resume data

• Storage of multiple analyses per user

• Scalable database design

This ensures stable performance even with increasing number of users and data.

3. Economic Feasibility

1. Development Cost

The project is economically feasible because:

• It uses open-source technologies

• No licensing cost is required

• Development tools are freely available

This makes the project cost-effective and suitable for academic purposes.

2. Operational Cost

Operational costs are minimal since:

• The system can run on standard hardware

• It can be hosted locally or on cloud platforms

• Maintenance requirements are low

3. Cost-Benefit Analysis

The benefits of the system outweigh the costs by:

• Reducing manual resume screening effort

• Improving efficiency in candidate evaluation

• Helping users improve resume quality

4. Operational Feasibility

1. User Acceptance

The system is designed to be:

• User-friendly

• Easy to navigate

• Quick in generating results

This increases acceptance among:

• Job seekers

• Students

• Entry-level candidates

2. Training Requirements

Minimal training is required because:

• The interface is simple and intuitive

• The workflow is straightforward

• Results are easy to understand

3. Workflow Compatibility

The system fits well into existing workflows by:

• Assisting resume preparation

• Supporting job application process

• Enhancing candidate evaluation

5. Schedule Feasibility

1. Development Timeline

The project can be completed within an academic timeline due to:

• Clearly defined modules

• Use of existing frameworks

• Incremental development approach

2. Risk Management

Potential risks such as parsing errors or incorrect analysis can be managed through:

• Data validation

• Testing with multiple resume formats

• Continuous improvement of logic

6. Legal and Ethical Feasibility

1. Data Privacy Compliance

The system ensures:

• Secure handling of user data

• Controlled access to information

• Protection of personal details

2. Ethical Evaluation Framework

The system:

• Does not make final hiring decisions

• Provides suggestions rather than judgments

• Avoids biased or unfair evaluation

All outputs are advisory in nature.

7. Social Feasibility

The project contributes to:

• Improving job readiness of users

• Helping students enhance resumes

• Promoting fair evaluation practices

It supports career development rather than replacing human decision-making.

8. Resource Feasibility

1. Human Resources

The project can be developed using:

• A small team of students

• Faculty guidance

• Basic software development skills

2. Hardware and Software Resources

Required resources include:

• Personal computers or laptops

• Development tools (IDE, database tools)

• Web browser and server

These resources are easily available in academic environments.

9. Feasibility Conclusion

Based on the technical, economic, operational, and ethical analysis, the Resume Analyzer System is highly feasible for implementation.

The project:

• Uses reliable and proven technologies

• Requires minimal cost and resources

• Is easy to develop and maintain

• Provides practical benefits to users

10. Summary

The feasibility study confirms that the project is:

• Technically implementable

• Economically affordable

• Operationally practical

• Ethically responsible

• Academically useful

Thus, the Resume Analyzer System is a viable and efficient solution for automated resume analysis and evaluation.

SYSTEM DESIGN:

1. Introduction to System Design

System Design defines how the Resume Analyzer System works internally and explains the architecture, components, data flow, and interaction between different modules.

The Resume Analyzer System is designed as a modular, scalable, and efficient web-based application that analyzes resumes by comparing them with job descriptions. The system evaluates resumes using keyword matching, skill analysis, and completeness checks to generate meaningful insights.

The design focuses on:

• Accuracy in resume evaluation

• Simplicity and user-friendliness

• Efficient data processing

• Easy integration and scalability

2. Overall System Architecture

The system follows a multi-tier architecture, ensuring proper separation of concerns and easy maintenance.

Architecture Type

Layered / Modular Architecture

Major Layers

Presentation Layer

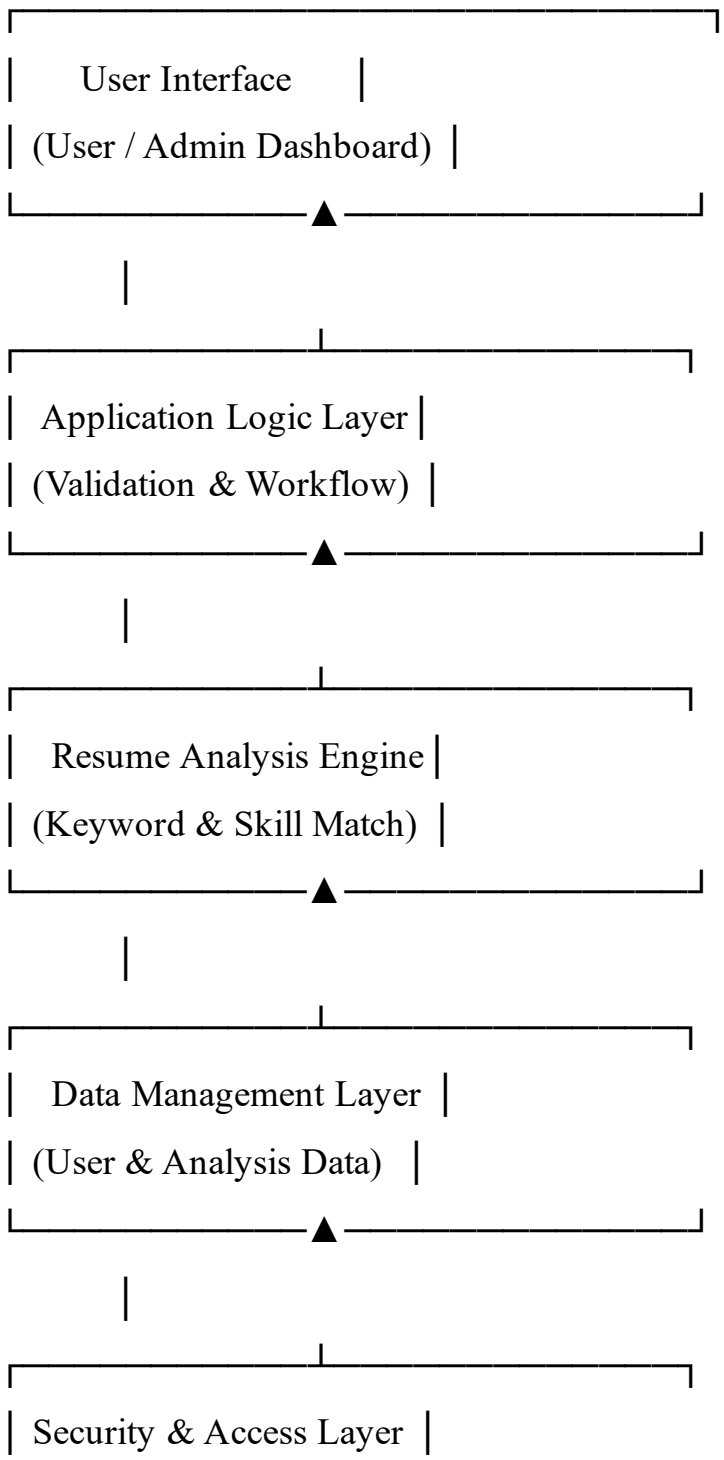
Application (Business Logic) Layer

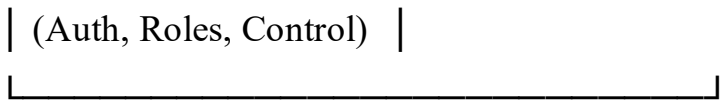
Analysis Engine Layer

Data Management Layer

Security & Access Control Layer

3. High-Level Architecture Diagram (Conceptual):





4. Module-Wise System Design:

1. Presentation Layer (User Interface Module)

Purpose

Provides interaction between users and the system.

Users

• Users (Job Seekers)

• Administrator

Functions

• User registration and login

• Resume upload and job description input

• Display of analysis results and scores

• History viewing

• Admin dashboard for user management

Characteristics

• Simple and clean UI

• Responsive design

• Easy navigation

• Visual representation of results

2. Application Logic Layer

Purpose

Acts as a bridge between the user interface and the analysis engine.

Responsibilities

• Request validation

• Workflow control

• Data preprocessing

• Communication between modules

Functions

• Handling user requests

• Processing resume upload

• Passing data to analysis module

• Managing errors and responses

3. Resume Analysis Engine

This is the core component of the system.

Sub-Modules

1. Resume Parsing Module

• Uses Apache Tika to extract text from PDF resumes

• Converts resume into readable text format

• Stores extracted content in database

2. Keyword Matching Module

• Extracts keywords from job description

• Removes unnecessary words

• Matches keywords with resume content

• Calculates keyword match percentage

3. Skill Matching Module

• Uses predefined skills dataset

• Identifies required skills from job description

• Compares with resume content

• Calculates skill match score

4. Completeness Evaluation Module

• Checks presence of important sections:

Education

Skills

Experience

Projects

• Calculates completeness score

5. Scoring and Recommendation Module

• Combines all scores to generate final score

• Identifies matched and missing skills

• Suggests relevant job roles

• Provides feedback for improvement

4. Data Management Layer

Purpose

Stores and manages all system data.

Data Types

• User details

• Resume content

• Analysis results

• Job descriptions

• Skills and recommendations

Database Characteristics

• Relational database structure

• Normalized tables

• Efficient data retrieval

• Secure storage

5. Security & Access Control Layer

Purpose

Ensures system security and controlled access.

Security Features

• User authentication and login

• Role-based access (User / Admin)

• Session management

• Secure data handling

Control Features

• Admin access control

• User suspend and activate functionality

• User deletion management

5. Data Flow Design

Step-by-Step Data Flow

User logs into the system

User uploads resume and enters job description

Resume is parsed using Apache Tika

Extracted text is sent to analysis engine

Keyword, skill, and completeness analysis is performed

Scores are calculated

Results are stored in database

Results are displayed on dashboard

6. Control Flow Design

• Input validation → Data processing → Analysis → Result generation

• System processes data step-by-step using rule-based logic

• Outputs are generated instantly after analysis

7. System Design Characteristics

1. Modularity

Each module works independently, making:

• Development easier

• Testing efficient

• Maintenance simple

2. Scalability

• Supports multiple users

• Can handle large number of analyses

• Ready for future enhancements

3. Reliability

• Proper error handling

• Data validation mechanisms

• Stable performance

4. Transparency

• Clear scoring logic

• Visible results and breakdown

• No hidden processing

8. Hardware & Software Design Requirements

Hardware Requirements

• Standard computer or laptop

• Minimum 4GB RAM

• Internet connectivity

Software Requirements

• Operating System: Windows / Linux

• Backend: Java (Spring Boot)

• Database: MySQL

• Frontend: HTML, CSS, Thymeleaf

9. Limitations in System Design

• No real-time resume ranking system

• Depends on quality of input resume and job description

• Limited to rule-based analysis

• No advanced AI or NLP-based understanding

10. System Design Summary

The system design of the Resume Analyzer System provides:

• A modular and layered architecture

• Efficient resume analysis mechanism

• Secure and scalable data handling

• User-friendly interface and workflow

This design ensures that the system is easy to implement, maintain, and extend, making it a practical solution for automated resume evaluation.

EXISTING APPLICATION (EXISTING SYSTEM):

1. Introduction to Existing System

The existing resume evaluation process in most recruitment systems is largely manual, fragmented, and inefficient. Recruiters typically review resumes individually and compare them with job descriptions based on personal judgment and experience. This process becomes time-consuming and inconsistent, especially when handling a large number of applications.

Most existing systems rely on basic filtering mechanisms or Applicant Tracking Systems (ATS), which focus on keyword-based screening without providing detailed insights or feedback to candidates. These systems lack transparency and do not help users understand how to improve their resumes.

2. Overview of Existing Applications

1. Manual Resume Screening

Recruiters manually review resumes to shortlist candidates.

• Reading resumes individually

• Comparing with job requirements

• Selecting candidates based on experience

Limitations:

• Time-consuming process

• Subjective decision-making

• Not scalable for large applications

• Inconsistent evaluation

2. Applicant Tracking Systems (ATS)

ATS systems are commonly used for resume filtering.

• Keyword-based filtering

• Resume parsing

• Candidate tracking

Limitations:

• Limited feedback to users

• No explanation of rejection

• Focus only on keyword presence

• No skill gap analysis

3. Online Job Portals

Platforms like LinkedIn, Naukri, and Indeed are used for job applications.

• Resume uploads

• Job searching

• Application tracking

Limitations:

• No detailed resume evaluation

• No scoring mechanism

• No personalized feedback

• No improvement suggestions

4. Basic Resume Builders

These tools help users create resumes.

• Resume templates

• Formatting assistance

• Content structuring

Limitations:

• Do not analyze resume quality

• No comparison with job description

• No skill matching

• No scoring or feedback

PROPOSED APPLICATION (PROPOSED SYSTEM):

1. Introduction to the Proposed System

The proposed application, Resume Analyzer System, is an intelligent web-based platform designed to overcome the limitations of existing resume evaluation methods. Unlike traditional systems that rely on manual screening or basic keyword filtering, this system provides automated analysis of resumes by comparing them with job descriptions.

The system evaluates resumes using rule-based techniques such as keyword matching, skill matching, and completeness checking. It generates meaningful insights, including match scores, skill gaps, and recommendations, helping users improve their resumes effectively.

2. Overview of the Proposed Application

The proposed system functions as a decision-support platform that:

• Analyzes resumes automatically

• Compares resumes with job descriptions

• Identifies matched and missing skills

• Generates detailed scores and feedback

• Maintains analysis history

The system assists users in improving their resumes and does not replace human decision-making in recruitment.

3. Key Features of the Proposed System

1. Resume Upload and Parsing

The system allows users to upload resumes and processes them using Apache Tika.

• PDF resume upload

• Text extraction from resume

• Storage of extracted content

• Handling multiple analyses

2. Resume Analysis Module

The system evaluates resumes using structured logic.

• Keyword matching with job description

• Skill matching using predefined dataset

• Completeness evaluation

• Final score generation

3. Skill Gap Identification Module

The system identifies differences between resume and job requirements.

• Detection of matched skills

• Identification of missing skills

• Suggestion for improvement

• Highlighting skill gaps

4. Recommendation Module

The system suggests job roles based on resume content.

• Role recommendations based on skills

• Mapping of skills to job roles

• Career guidance support

5. Analysis History and Visualization

The system maintains and displays past records.

• Storage of previous analyses

• Viewing analysis history

• Display of scores and results

• Graphical representation of trends

4. User Roles in the Proposed Application

1. Users (Job Seekers)

Users can:

• Register and login

• Upload resumes

• Enter job descriptions

• View analysis results

• Access history

2. Administrator (Admin)

Admin can:

• Login using predefined credentials

• View all users

• Monitor analysis data

• Suspend and activate users

• Delete user accounts

5**. Advantages of the Proposed System Over Existing System**

Aspect

Existing System

Proposed System

Evaluation Method

Manual / Basic Filtering

Automated Analysis

Feedback

Limited

Detailed Feedback

Skill Analysis

Not available

Available

Transparency

Low

High

User Improvement

Not supported

Supported

6. Ethical and Security Considerations

The proposed system ensures:

• Secure handling of user data

• Controlled access to system features

• No automatic decision-making

• Advisory-based analysis

The system provides suggestions and does not replace human judgment.

7. System Workflow of Proposed Application

User logs into the system

Resume is uploaded

Job description is entered

Resume is parsed using Apache Tika

Analysis is performed (keyword, skill, completeness)

Scores and feedback are generated

Results are displayed to the user

8. Implementation Environment

Software Environment

• Backend: Java (Spring Boot)

• Frontend: HTML, CSS, Thymeleaf

• Database: MySQL

Hardware Environment

• Standard computer or server

• Minimum 4GB RAM

• Internet connectivity

9. Limitations of the Proposed Application

• Limited to rule-based analysis

• No advanced AI or NLP processing

• Depends on quality of input data

• No real-time job portal integration

10. Future Enhancement Support

The system can be extended to include:

• AI-based resume analysis

• NLP-based semantic matching

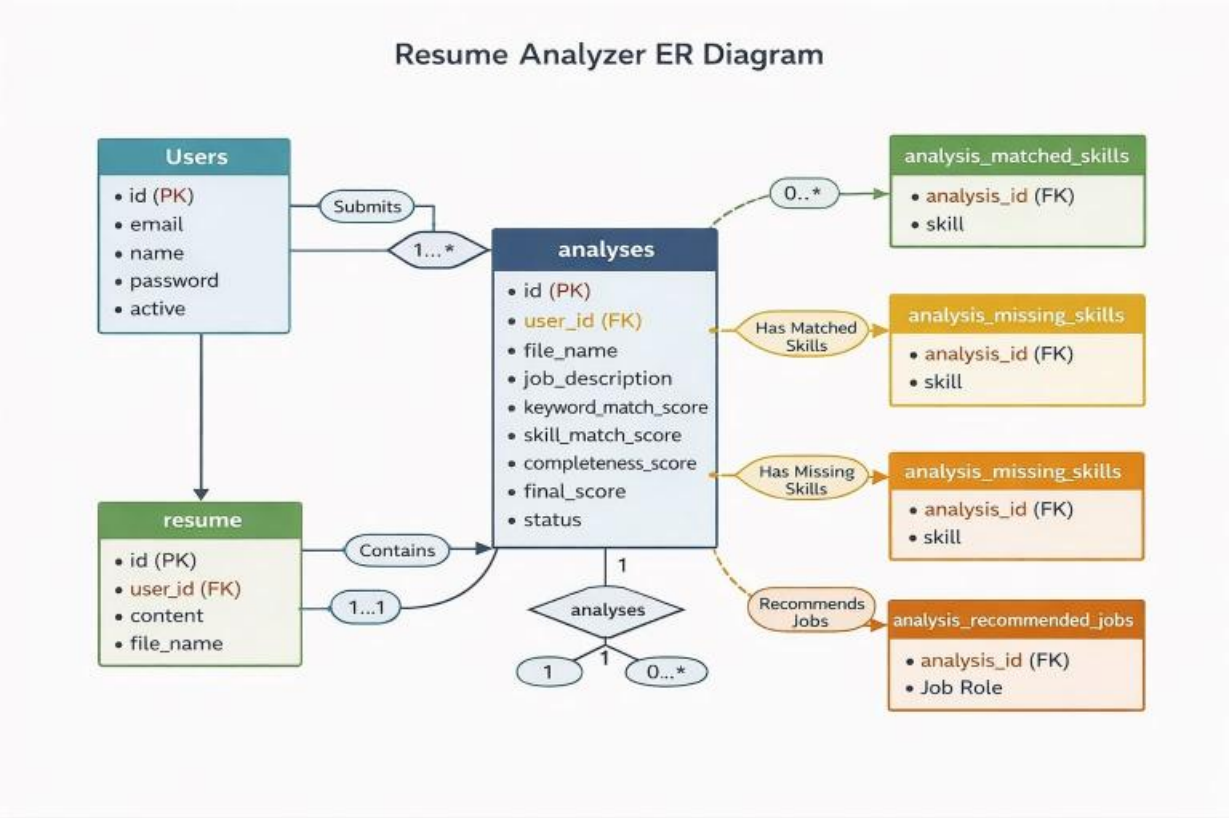
• Integration with job portals

• Advanced analytics and dashboards

11. Summary of Proposed Application

The Resume Analyzer System provides an automated, efficient, and user-friendly solution for resume evaluation. It enhances the recruitment process by offering detailed analysis, feedback, and recommendations while maintaining simplicity and scalability.

ER DIAGRAM


Resume Analyzer ER Diagram
Users
• id (PK)
• email
• name
• password
• active
Submits
1...*
analyses
• id (PK)
• user_id (FK)
• file_name
• job_description
• keyword_match_score
• skill_match_score
• completeness_score
• final_score
• status
0..*
analysis_matched_skills
• analysis_id (FK)
• skill
Has Matched Skills
analysis_missing_skills
• analysis_id (FK)
• skill
Has Missing Skills
analysis_missing_skills
• analysis_id (FK)
• skill
Recommends Jobs
analysis_recommended_jobs
• analysis_id (FK)
• Job Role
resume
• id (PK)
• user_id (FK)
• content
• file_name
Contains
1...1
1
analyses
1
1
0...*

Screenshot of database

```
CREATE DATABASE IF NOT EXISTS resume_analyzer_db
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE resume_analyzer_db;

-- Drop tables if exist (order matters because of FK)
DROP TABLE IF EXISTS analysis_recommended_jobs;
DROP TABLE IF EXISTS analysis_missing_skills;
DROP TABLE IF EXISTS analysis_matched_skills;
DROP TABLE IF EXISTS analyses;
DROP TABLE IF EXISTS resume;
DROP TABLE IF EXISTS users;

-- 1. USERS TABLE

CREATE TABLE users (
    id            BIGINT PRIMARY KEY AUTO_INCREMENT,
    name          VARCHAR(150) NOT NULL,
    email         VARCHAR(150) NOT NULL UNIQUE,
    password      VARCHAR(255) NOT NULL,
    active        BOOLEAN DEFAULT TRUE
);
```

```
-- 2. RESUME TABLE

CREATE TABLE resume (
    id              BIGINT PRIMARY KEY AUTO_INCREMENT,
    user_id         BIGINT NOT NULL,
    file_name       VARCHAR(255),
    content         LONGTEXT,
    score           DECIMAL(5,2),

    CONSTRAINT fk_resume_user
        FOREIGN KEY (user_id) REFERENCES users(id)
        ON DELETE CASCADE
);

-- 3. ANALYSES TABLE (MAIN TABLE)

CREATE TABLE analyses (
    id                  BIGINT PRIMARY KEY AUTO_INCREMENT,
    user_id             BIGINT NOT NULL,
    file_name           VARCHAR(255),
    job_description     TEXT,

    keyword_match_score   DECIMAL(5,2),
    skill_match_score     DECIMAL(5,2),
    completeness_score    DECIMAL(5,2),
    final_score           DECIMAL(5,2),
```

```
    status              VARCHAR(50), -- Good / Moderate / Poor
    insight             TEXT,
    created_at          TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT fk_analysis_user
        FOREIGN KEY (user_id) REFERENCES users(id)
        ON DELETE CASCADE
);

-- 4. MATCHED SKILLS TABLE

CREATE TABLE analysis_matched_skills (
    id          BIGINT PRIMARY KEY AUTO_INCREMENT,
    analysis_id   BIGINT NOT NULL,
    skill       VARCHAR(100),

    CONSTRAINT fk_matched_analysis
        FOREIGN KEY (analysis_id) REFERENCES analyses(id)
        ON DELETE CASCADE
);

5. MISSING SKILLS TABLE

CREATE TABLE analysis_missing_skills (
    id          BIGINT PRIMARY KEY AUTO_INCREMENT,
    analysis_id   BIGINT NOT NULL,
    skill       VARCHAR(100),
```

```
    CONSTRAINT fk_missing_analysis
        FOREIGN KEY (analysis_id) REFERENCES analyses(id)
        ON DELETE CASCADE
);
```

6. RECOMMENDED JOB ROLES TABLE

```
CREATE TABLE analysis_recommended_jobs (
    id            BIGINT PRIMARY KEY AUTO_INCREMENT,
    analysis_id   BIGINT NOT NULL,
    job_role      VARCHAR(150),

    CONSTRAINT fk_jobs_analysis
        FOREIGN KEY (analysis_id) REFERENCES analyses(id)
        ON DELETE CASCADE
);
```

Code :

Controller file:

AdminController.java

```
package com.example.ResumeAnalyzerPro_Final.controller;

import com.example.ResumeAnalyzerPro_Final.entity.Analysis;
import com.example.ResumeAnalyzerPro_Final.entity.User;
import com.example.ResumeAnalyzerPro_Final.service.ResumeService;
import com.example.ResumeAnalyzerPro_Final.service.UserService;
import jakarta.servlet.http.HttpSession;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Page;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Map;
import java.util.HashMap;

@Controller
@RequestMapping("/admin")
public class AdminController {
```

```
@Autowired
private UserService userService;

@Autowired
private ResumeService resumeService;

private boolean isAdmin(HttpSession session) {
    Boolean isAdminFlag = (Boolean) session.getAttribute("admin");
    if (Boolean.TRUE.equals(isAdminFlag)) {
        return true;
    }
    User user = (User) session.getAttribute("user");
    return user != null && "ADMIN".equals(user.getRole());
}

private String getAdminName(HttpSession session) {
    String adminName = (String) session.getAttribute("adminName");
    if (adminName != null && !adminName.isBlank()) {
        return adminName;
    }
    User user = (User) session.getAttribute("user");
    if (user != null && "ADMIN".equals(user.getRole())) {
        return user.getName();
    }
    return "Administrator";
}
```

```
    @GetMapping("/login")
    public String loginPage(HttpSession session, Model model) {
        if (isAdmin(session)) {
            return "redirect:/admin/dashboard";
        }

        // Check if admin account exists
        List<User> admins = userService.getAllUsers().stream()
                .filter(u -> "ADMIN".equals(u.getRole()))
                .toList();

        if (admins.isEmpty()) {
            model.addAttribute("error",
                    "No admin account found. Please register an admin account with email 'admin@admin.com' first.");
        }

        return "admin-login";
    }

    @PostMapping("/login")
    public String login(@RequestParam String email, @RequestParam String password, HttpSession session, Model model) {
        User admin = userService.loginAdmin(email, password);
        if (admin != null) {
            session.setAttribute("user", admin);
            return "redirect:/admin/dashboard";
        }
```

```
        model.addAttribute("error",
                "Invalid admin credentials. Please register an admin account with
email 'admin@admin.com' first.");
        return "admin-login";
    }

    @GetMapping("/dashboard")
    public String dashboard(HttpSession session, Model model) {
        if (!isAdmin(session)) {
            return "redirect:/admin/login";
        }

        List<User> users = userService.getAllUsers();
        List<Analysis> analyses = resumeService.getAllAnalyses();

        model.addAttribute("users", users);
        model.addAttribute("analyses", analyses);
        model.addAttribute("adminName", getAdminName(session));
        model.addAttribute("admin", session.getAttribute("user"));
        return "admin-dashboard";
    }

    @GetMapping("/user/{id}")
    public String viewUserDetails(@PathVariable Long id,
            @RequestParam(value = "page", defaultValue = "0") int page,
            HttpSession session,
            Model model) {
```

```
        if (!isAdmin(session)) {
            return "redirect:/admin/login";
        }

        User selectedUser = userService.getUserById(id);
        if (selectedUser == null) {
            return "redirect:/admin/dashboard";
        }

        Page<Analysis> analysisPage =
resumeService.getHistoryPage(selectedUser, page);

        List<com.example.ResumeAnalyzerPro_Final.entity.Resume> resumes =
resumeService.getResumesByUser(selectedUser);
        Map<String, String> resumeContents = new HashMap<>();
        for (com.example.ResumeAnalyzerPro_Final.entity.Resume r : resumes) {
            resumeContents.put(r.getFileName(), r.getContent());
        }

        model.addAttribute("userDetails", selectedUser);
        model.addAttribute("resumes", resumes);
        model.addAttribute("analysisPage", analysisPage);
        model.addAttribute("currentPage", page);
        model.addAttribute("hasMore", analysisPage.hasNext());
        model.addAttribute("resumeContents", resumeContents);
        model.addAttribute("adminName", getAdminName(session));
        return "admin-user-details";
    }
```

```
    @GetMapping("/user/{userId}/analysis/{analysisId}")
    public String viewUserAnalysis(@PathVariable Long userId,
            @PathVariable Long analysisId,
            HttpSession session,
            Model model) {
        if (!isAdmin(session)) {
            return "redirect:/admin/login";
        }

        User selectedUser = userService.getUserById(userId);
        if (selectedUser == null) {
            return "redirect:/admin/dashboard";
        }

        Analysis analysis = resumeService.getAnalysisById(analysisId);
        if (analysis == null || analysis.getUser() == null ||
!analysis.getUser().getId().equals(userId)) {
            return "redirect:/admin/user/" + userId;
        }

        List<com.example.ResumeAnalyzerPro_Final.entity.Resume> resumes =
resumeService.getResumesByUser(selectedUser);
        Map<String, String> resumeContents = new HashMap<>();
        for (com.example.ResumeAnalyzerPro_Final.entity.Resume r : resumes) {
            resumeContents.put(r.getFileName(), r.getContent());
        }
```

```
        model.addAttribute("userDetails", selectedUser);
        model.addAttribute("analysis", analysis);
        model.addAttribute("resumeContents", resumeContents);
        model.addAttribute("adminName", getAdminName(session));
        return "admin-analysis-details";
    }

    @PostMapping("/user/{id}/delete")
    public String deleteUser(@PathVariable Long id, HttpSession session) {
        if (!isAdmin(session)) {
            return "redirect:/admin/login";
        }

        User currentUser = (User) session.getAttribute("user");
        if (currentUser != null && currentUser.getId() != null &&
currentUser.getId().equals(id)) {
            return "redirect:/admin/dashboard";
        }

        userService.deleteUser(id);
        return "redirect:/admin/dashboard";
    }

    @PostMapping("/user/{id}/toggle-lock")
    public String toggleLockUser(@PathVariable Long id, HttpSession session) {
        if (!isAdmin(session)) {
            return "redirect:/admin/login";
        }
```

```
        User admin = (User) session.getAttribute("user");
        if (admin == null || admin.getId() == null || !admin.getId().equals(id)) {
            userService.toggleUserLock(id);
        }
        return "redirect:/admin/dashboard";
    }

    @GetMapping("/logout")
    public String logout(HttpSession session) {
        session.invalidate();
        return "redirect:/admin/login";
    }
}
```

AuthController.java

```
package com.example.ResumeAnalyzerPro_Final.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;

import jakarta.servlet.http.HttpSession;

import com.example.ResumeAnalyzerPro_Final.entity.User;
```

```
import com.example.ResumeAnalyzerPro_Final.service.UserService;

@Controller
public class AuthController {

    @Autowired
    private UserService service;

    @GetMapping("/")
    public String home(HttpSession session) {
        return "landing";
    }

    @GetMapping("/login")
    public String login(HttpSession session) {
        if (session.getAttribute("user") != null) {
            return "redirect:/dashboard";
        }
        return "login";
    }

    @GetMapping("/register")
    public String register(HttpSession session) {
        if (session.getAttribute("user") != null) {
            return "redirect:/dashboard";
        }
        return "register";
```

```
}

@PostMapping("/register")
public String register(User user, Model model) {

    try {
        service.register(user);
        model.addAttribute("success", "Registration successful. Please log in.");
        return "login";
    } catch (RuntimeException ex) {
        model.addAttribute("error", ex.getMessage());
        model.addAttribute("user", user);
        return "register";
    }

}

@PostMapping("/login")
public String login(String email, String password,
        HttpSession session,
        Model model) {

    User user = service.login(email, password);

    if (user == null) {

        model.addAttribute("error", "Invalid email or password");
```

```
            return "login";

        }

        if (user.isLocked()) {
            model.addAttribute("error", "Your account has been suspended by an
administrator.");
            return "login";
        }

        session.setAttribute("user", user);

        if ("ADMIN".equals(user.getRole())) {
            return "redirect:/admin/dashboard";
        }

        return "redirect:/dashboard";

    }

    @GetMapping("/logout")
    public String logout(HttpSession session) {
        session.invalidate();
        return "redirect:/login";
    }

}
```

ResumeController.java

```
package com.example.ResumeAnalyzerPro_Final.controller;

import com.example.ResumeAnalyzerPro_Final.entity.Analysis;
import com.example.ResumeAnalyzerPro_Final.entity.Resume;
import com.example.ResumeAnalyzerPro_Final.entity.User;
import com.example.ResumeAnalyzerPro_Final.service.ResumeService;
import com.itextpdf.kernel.colors.ColorConstants;
import com.itextpdf.kernel.pdf.PdfDocument;
import com.itextpdf.kernel.pdf.PdfWriter;
import com.itextpdf.layout.Document;
import com.itextpdf.layout.element.ListItem;
import com.itextpdf.layout.element.Paragraph;
import com.itextpdf.layout.element.Text;
import jakarta.servlet.http.HttpSession;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Page;
import org.springframework.http.ContentDisposition;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
```

```
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.ResponseBody;
import org.springframework.web.multipart.MultipartFile;

import java.io.ByteArrayOutputStream;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.List;

@Controller
public class ResumeController {

    @Autowired
    private ResumeService service;

    @GetMapping("/dashboard")
    public String dashboard(HttpSession session,
            Model model) {
        User user = (User) session.getAttribute("user");
        if (user == null) {
            return "redirect:/login";
        }
        if ("ADMIN".equals(user.getRole())) {
            return "redirect:/admin/dashboard";
        }
```

```
        populateDashboardModel(model, user, service.getLatestAnalysis(user));
        return "dashboard";
    }

    @PostMapping("/analyze")
    public String analyze(@RequestParam(value = "file", required = false)
MultipartFile file,
            @RequestParam(value = "resumeContent", required = false) String
resumeContent,
            @RequestParam(value = "jobDesc", required = false) String jobDesc,
            HttpSession session,
            Model model) {

        User user = (User) session.getAttribute("user");
        if (user == null) {
            return "redirect:/login";
        }

        try {
            String normalizedJobDesc = jobDesc == null ? "" : jobDesc.trim();
            String normalizedResumeContent = resumeContent == null ? "" :
resumeContent.trim();

            Resume resume = null;
            if (file != null && !file.isEmpty()) {
                resume = service.upload(file, user);
                if (resume == null) {
```

```
                populateDashboardModel(model, user,
service.getLatestAnalysis(user));
                model.addAttribute("error",
                        "File processing failed. Please try a different file format or
paste text instead.");
                return "dashboard";
            }
            normalizedResumeContent = resume.getContent();
        } else if (!normalizedResumeContent.isBlank()) {
            resume = service.saveTextResume(normalizedResumeContent, user);
        }


        if (normalizedResumeContent.isBlank() || normalizedJobDesc.isBlank())
{
            populateDashboardModel(model, user,
service.getLatestAnalysis(user));
            model.addAttribute("error",
                    "Please upload a resume or paste resume text, and provide job
description.");
            return "dashboard";
        }


        int keywordMatch =
service.calculateKeywordMatchScore(normalizedResumeContent,
normalizedJobDesc);
        int trackedSkills =
service.calculateTrackedSkillsScore(normalizedResumeContent,
normalizedJobDesc);
        int completeness =
service.calculateCompletenessScore(normalizedResumeContent);
```

```
        int finalScore = service.calculateFinalScore(keywordMatch,
trackedSkills, completeness);


        List<String> matchedSkills =
service.findMatchedSkills(normalizedResumeContent, normalizedJobDesc);
        List<String> missingSkills =
service.findMissingSkills(normalizedResumeContent, normalizedJobDesc);
        List<String> recommendedJobs =
service.recommendJobs(normalizedResumeContent, normalizedJobDesc);
        String fileName;


        if (file != null && !file.isEmpty()) {
            fileName = file.getOriginalFilename(); // uploaded file name
        } else {
            fileName = "pasted-resume.txt"; // fallback
        }
        Analysis analysis = service.saveAnalysis(
                user,
                finalScore,
                keywordMatch,
                trackedSkills,
                completeness,
                matchedSkills,
                missingSkills,
                recommendedJobs,
                fileName,
                normalizedJobDesc);


        populateDashboardModel(model, user, analysis);
```

```
            model.addAttribute("resumeText", normalizedResumeContent);
            model.addAttribute("jobDesc", normalizedJobDesc);
            model.addAttribute("sourceFileName", resume != null ?
resume.getFileName() : "pasted-resume.txt");
            return "dashboard";

        } catch (Exception e) {
            populateDashboardModel(model, user, service.getLatestAnalysis(user));
            model.addAttribute("error",
                    "An error occurred during analysis. Please try again. Error: " +
e.getMessage());
            return "dashboard";
        }
    }

    @GetMapping("/analysis/{id}/view")
    public String viewAnalysis(@PathVariable Long id,
            HttpSession session,
            Model model) {
        User user = (User) session.getAttribute("user");
        if (user == null) {
            return "redirect:/login";
        }

        Analysis analysis = service.getAnalysisByIdForUser(id, user);
        if (analysis == null) {
            populateDashboardModel(model, user, service.getLatestAnalysis(user));
            model.addAttribute("error", "Analysis not found.");
```

```
        return "dashboard";
    }

    populateDashboardModel(model, user, analysis);
    return "dashboard";
}

@GetMapping("/history/more")
@ResponseBody
public ResponseEntity<HistoryChunkResponse> loadMoreHistory(
        @RequestParam(value = "offset", defaultValue = "10") int offset,
        HttpSession session) {
    User user = (User) session.getAttribute("user");
    if (user == null) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).build();
    }

    int safeOffset = Math.max(0, offset);
    int pageSize = ResumeService.HISTORY_PAGE_SIZE;
    int pageIndex = safeOffset / pageSize;

    Page<Analysis> chunk = service.getHistoryPage(user, pageIndex);
    List<HistoryItem> items = chunk.getContent().stream()
            .map(analysis -> new HistoryItem(
                    analysis.getId(),
                    analysis.getFinalScore(),
                    analysis.getStatus(),
```

```
                analysis.getFileName(),
                analysis.getInsight(),
                analysis.getCreatedAt().format(DateTimeFormatter.ofPattern("dd
MMM yyyy, hh:mm a"))))
            .toList();

    int loadedCount = safeOffset + items.size();
    boolean hasMore = loadedCount < service.getTotalAnalyses(user);
    return ResponseEntity.ok(new HistoryChunkResponse(items, hasMore,
loadedCount));
  }

  @GetMapping("/analysis/{id}/pdf")
  public ResponseEntity<byte[]> downloadPdf(@PathVariable Long id,
HttpSession session) {
    User user = (User) session.getAttribute("user");
    if (user == null) {
      return ResponseEntity.status(HttpStatus.FOUND)
          .header(HttpHeaders.LOCATION, "/login")
          .build();
    }

    Analysis analysis = service.getAnalysisByIdForUser(id, user);
    if (analysis == null) {
      return ResponseEntity.notFound().build();
    }

    byte[] pdfBytes = buildAnalysisPdf(analysis, user.getName());
```

```
        String filename = "analysis-" + analysis.getId() + ".pdf";

        HttpHeaders headers = new HttpHeaders();
        headers.setContentType(MediaType.APPLICATION_PDF);
headers.setContentDisposition(ContentDisposition.attachment().filename(filena
me).build());

        return new ResponseEntity<>(pdfBytes, headers, HttpStatus.OK);
    }

    private void populateDashboardModel(Model model, User user, Analysis
selectedAnalysis) {
        Page<Analysis> historyPageData = service.getHistoryPage(user, 0);
        List<Analysis> history = historyPageData.getContent();
        Analysis latest = service.getLatestAnalysis(user);
        long totalAnalyses = service.getTotalAnalyses(user);

        Analysis active = selectedAnalysis != null ? selectedAnalysis : latest;

        model.addAttribute("user", user);
        model.addAttribute("totalAnalyses", totalAnalyses);
        model.addAttribute("latestScore", latest != null ? latest.getFinalScore() :
0);
        model.addAttribute("latestStatus", latest != null ? latest.getStatus() : "No
Analysis Yet");
        model.addAttribute("recommendedJobsCount", latest != null ?
latest.getRecommendedJobs().size() : 0);
```

```
        model.addAttribute("history", history);
        model.addAttribute("historyLoadedCount", history.size());
        model.addAttribute("historyHasMore", history.size() < totalAnalyses);
        model.addAttribute("activeAnalysis", active);

        List<Analysis> graphHistory = service.getLatestAnalyses(user, 10);
        List<String> graphLabels = new ArrayList<>();
        List<Integer> graphScores = new ArrayList<>();
        int graphSize = graphHistory.size();
        int startRun = Math.max(1, (int) (totalAnalyses - graphSize + 1));
        for (int i = graphSize - 1; i >= 0; i--) {
            Analysis analysis = graphHistory.get(i);
            graphLabels.add("Run " + (startRun + (graphSize - 1 - i)));
            graphScores.add(analysis.getFinalScore());
        }

        model.addAttribute("graphLabels", graphLabels);
        model.addAttribute("graphScores", graphScores);
        model.addAttribute("dateFormatter", DateTimeFormatter.ofPattern("dd
MMM yyyy, hh:mm a"));
    }

    private byte[] buildAnalysisPdf(Analysis analysis, String userName) {
        ByteArrayOutputStream outputStream = new ByteArrayOutputStream();
        PdfWriter writer = new PdfWriter(outputStream);
        PdfDocument pdfDocument = new PdfDocument(writer);
        Document document = new Document(pdfDocument);
```

```
        document.add(new Paragraph("Resume Analyzer Pro - Analysis Report")
                .setFontSize(18)
                .setBold());

        document.add(new Paragraph("Candidate: " + userName));
        document.add(new Paragraph(
                "Generated: " +
analysis.getCreatedAt().format(DateTimeFormatter.ofPattern("dd MMM yyyy,
hh:mm a"))));

        document.add(new Paragraph(" "));
        document.add(new Paragraph("Final Score: " + analysis.getFinalScore() +
"%")
                .setFontSize(14)
                .setBold()
                .setFontColor(ColorConstants.BLUE));

        document.add(new Paragraph("Status: " + analysis.getStatus()));

        document.add(new Paragraph(" "));
        document.add(new Paragraph("Job Description").setBold());
        document.add(new Paragraph(
                analysis.getJobDescription() == null ||
analysis.getJobDescription().isBlank()
                        ? "No job description provided."
                        : analysis.getJobDescription()));

        document.add(new Paragraph(" "));
        document.add(new Paragraph("Score Breakdown").setBold());
```

```
        document.add(new Paragraph("Keyword Match: " +
analysis.getKeywordMatchScore() + "%"));
        document.add(new Paragraph("Tracked Skills: " +
analysis.getSkillMatchScore() + "%"));
        document.add(new Paragraph("Completeness: " +
analysis.getCompletenessScore() + "%"));


        document.add(new Paragraph(" "));
        document.add(new Paragraph("Matched Skills").setBold());
        document.add(buildList(analysis.getMatchedSkills(), "No matched
skills."));


        document.add(new Paragraph(" "));
        document.add(new Paragraph("Missing Skills").setBold());
        document.add(buildList(analysis.getMissingSkills(), "No missing skills."));


        document.add(new Paragraph(" "));
        document.add(new Paragraph("Recommended Jobs").setBold());
        document.add(buildList(analysis.getRecommendedJobs(), "No
recommended jobs."));


        document.close();
        return outputStream.toByteArray();
    }


    private com.itextpdf.layout.element.IBlockElement buildList(List<String>
values, String emptyText) {
        if (values == null || values.isEmpty()) {
            return new Paragraph(new Text(emptyText));
```

```
        }

        com.itextpdf.layout.element.List list = new
com.itextpdf.layout.element.List();
        for (String value : values) {
            list.add(new ListItem(value));
        }
        return list;
    }

    private record HistoryChunkResponse(List<HistoryItem> items, boolean
hasMore, int loadedCount) {
    }

    private record HistoryItem(Long id,
            int finalScore,
            String status,
            String fileName,
            String insight,
            String createdAt) {
    }
}
```

Entity

Analyis.java

```
package com.example.ResumeAnalyzerPro_Final.entity;

import jakarta.persistence.CollectionTable;
import jakarta.persistence.Column;
import jakarta.persistence.ElementCollection;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.PrePersist;
import jakarta.persistence.Table;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "analyses")
public class Analysis {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```
    private int finalScore;
    private int keywordMatchScore;
    private int skillMatchScore;
    private int completenessScore;
    private String status;
    private String insight;
    private String fileName;
    @Column(columnDefinition = "TEXT")
    private String jobDescription;


    @ElementCollection(fetch = FetchType.EAGER)
    @CollectionTable(name = "analysis_matched_skills", joinColumns =
@JoinColumn(name = "analysis_id"))
    @Column(name = "skill")
    private List<String> matchedSkills = new ArrayList<>();


    @ElementCollection(fetch = FetchType.EAGER)
    @CollectionTable(name = "analysis_missing_skills", joinColumns =
@JoinColumn(name = "analysis_id"))
    @Column(name = "skill")
    private List<String> missingSkills = new ArrayList<>();


    @ElementCollection(fetch = FetchType.EAGER)
    @CollectionTable(name = "analysis_recommended_jobs", joinColumns =
@JoinColumn(name = "analysis_id"))
    @Column(name = "job_role")
    private List<String> recommendedJobs = new ArrayList<>();
```

```
private LocalDateTime createdAt;

@ManyToOne
private User user;

@PrePersist
public void prePersist() {
    if (createdAt == null) {
        createdAt = LocalDateTime.now();
    }
}

public Long getId() {
    return id;
}

public void setId(Long id) {
    this.id = id;
}

public int getFinalScore() {
    return finalScore;
}

public void setFinalScore(int finalScore) {
    this.finalScore = finalScore;
}
```

```
public int getKeywordMatchScore() {
    return keywordMatchScore;
}

public void setKeywordMatchScore(int keywordMatchScore) {
    this.keywordMatchScore = keywordMatchScore;
}

public int getSkillMatchScore() {
    return skillMatchScore;
}

public void setSkillMatchScore(int skillMatchScore) {
    this.skillMatchScore = skillMatchScore;
}

public int getCompletenessScore() {
    return completenessScore;
}

public void setCompletenessScore(int completenessScore) {
    this.completenessScore = completenessScore;
}

public String getStatus() {
    return status;
```

```
}

public void setStatus(String status) {
    this.status = status;
}

public List<String> getMatchedSkills() {
    return matchedSkills;
}

public void setMatchedSkills(List<String> matchedSkills) {
    this.matchedSkills = matchedSkills;
}

public List<String> getMissingSkills() {
    return missingSkills;
}

public void setMissingSkills(List<String> missingSkills) {
    this.missingSkills = missingSkills;
}

public List<String> getRecommendedJobs() {
    return recommendedJobs;
}

public void setRecommendedJobs(List<String> recommendedJobs) {
```

```
    this.recommendedJobs = recommendedJobs;
}

public LocalDateTime getCreatedAt() {
    return createdAt;
}

public void setCreatedAt(LocalDateTime createdAt) {
    this.createdAt = createdAt;
}

public User getUser() {
    return user;
}

public void setUser(User user) {
    this.user = user;
}

public String getInsight() {
    return insight;
}

public void setInsight(String insight) {
    this.insight = insight;
}
```

```
    public String getFileName() {
        return fileName;
    }

    public void setFileName(String fileName) {
        this.fileName = fileName;
    }

    public String getJobDescription() {
        return jobDescription;
    }

    public void setJobDescription(String jobDescription) {
        this.jobDescription = jobDescription;
    }
}
```

Resume.java

```
package com.example.ResumeAnalyzerPro_Final.entity;

import jakarta.persistence.*;

@Entity
public class Resume {
```

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

private String fileName;

@Lob
@Column(columnDefinition = "LONGTEXT")
private String content;

private int score;

@ManyToOne
private User user;

public Long getId() {
    return id;
}

public void setId(Long id) {
    this.id = id;
}

public String getFileName() {
    return fileName;
}
```

```
public void setFileName(String fileName) {
    this.fileName = fileName;
}

public String getContent() {
    return content;
}

public void setContent(String content) {
    this.content = content;
}

public int getScore() {
    return score;
}

public void setScore(int score) {
    this.score = score;
}

public User getUser() {
    return user;
}

public void setUser(User user) {
    this.user = user;
}
```

}

User.java

```
package com.example.ResumeAnalyzerPro_Final.entity;

import jakarta.persistence.*;

@Entity
@Table(name="users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @Column(unique = true)
    private String email;

    private String password;

    @Column(nullable = false)
    private String role = "USER";
```

```
@Column(nullable = false)
private boolean locked = false;

public Long getId() {
    return id;
}

public void setId(Long id) {
    this.id = id;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}
```

```
    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public String getRole() {
        return (role == null || role.trim().isEmpty()) ? "USER" : role;
    }

    public void setRole(String role) {
        this.role = role;
    }

    public boolean isLocked() {
        return locked;
    }

    public void setLocked(boolean locked) {
        this.locked = locked;
    }
}
```

Repository

AnalysisRepository.java

```
package com.example.ResumeAnalyzerPro_Final.repository;

import com.example.ResumeAnalyzerPro_Final.entity.Analysis;
import com.example.ResumeAnalyzerPro_Final.entity.User;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;
import java.util.Optional;

public interface AnalysisRepository extends JpaRepository<Analysis, Long> {

    List<Analysis> findByUserOrderByCreatedAtDesc(User user);

    Page<Analysis> findByUserOrderByCreatedAtDesc(User user, Pageable
pageable);

    Optional<Analysis> findFirstByUserOrderByCreatedAtDesc(User user);

    Optional<Analysis> findByIdAndUser(Long id, User user);

    long countByUser(User user);

    void deleteByUser(User user);
}
```

ResumeRepository.java

```
package com.example.ResumeAnalyzerPro_Final.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.example.ResumeAnalyzerPro_Final.entity.Resume;

import com.example.ResumeAnalyzerPro_Final.entity.User;

public interface ResumeRepository extends JpaRepository<Resume, Long> {

    void deleteByUser(User user);

    java.util.List<Resume> findByUser(User user);
}
```

UserRepository.java

```
package com.example.ResumeAnalyzerPro_Final.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.example.ResumeAnalyzerPro_Final.entity.User;

public interface UserRepository extends JpaRepository<User, Long> {
```

```
    User findFirstByEmail(String email);

    User findFirstByEmailIgnoreCase(String email);

    User findFirstByEmailIgnoreCaseAndRole(String email, String role);

}
```

Service

ResumeService.java

```
package com.example.ResumeAnalyzerPro_Final.service;

import com.example.ResumeAnalyzerPro_Final.entity.Analysis;
import com.example.ResumeAnalyzerPro_Final.entity.Resume;
import com.example.ResumeAnalyzerPro_Final.entity.User;
import com.example.ResumeAnalyzerPro_Final.repository.AnalysisRepository;
import com.example.ResumeAnalyzerPro_Final.repository.ResumeRepository;
import com.example.ResumeAnalyzerPro_Final.util.TikaParser;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.stereotype.Service;
import org.springframework.web.multipart.MultipartFile;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;

@Service
public class ResumeService {

    public static final int HISTORY_PAGE_SIZE = 10;

    @Autowired
    private ResumeRepository repo;

    @Autowired
    private TikaParser parser;

    @Autowired
    private AnalysisRepository analysisRepository;

    // Core skills (for basic resume score)
    private final List<String> coreSkills = Arrays.asList(
            "java", "python", "mysql", "html", "css");
```

```
// Normalize text (remove punctuation)
private String normalize(String text) {
    if (text == null)
        return "";

    return text.toLowerCase()
            .replaceAll("[^a-z0-9+#. ]", " ") // remove punctuation
            .replaceAll("\\s+", " ") // remove extra spaces
            .trim();
}

private boolean containsWord(String text, String skill) {
    if (text == null || skill == null)
        return false;

    String normalizedText = " " + text + " ";
    String normalizedSkill = skill.toLowerCase();

    // handle plural (api vs apis)
    if (normalizedText.contains(" " + normalizedSkill + " ") ||
            normalizedText.contains(" " + normalizedSkill + "s ")) {
        return true;
    }

    return false;
}
```

```
    // All skills (for matching and recommendations)
    private final List<String> allSkills = loadSkills();

    public List<String> loadSkills() {

        try (java.io.InputStream is =
getClass().getClassLoader().getResourceAsStream("skills.txt")) {

            if (is == null) {
                return Arrays.asList("java", "python", "mysql", "html", "css");
            }

            try (java.io.BufferedReader reader = new java.io.BufferedReader(new
java.io.InputStreamReader(is))) {

                return reader.lines()
                        .map(String::trim)
                        .filter(line -> !line.isBlank())
                        .map(String::toLowerCase)
                        .collect(Collectors.toList());
            }

        } catch (Exception e) {
            return Arrays.asList("java", "python", "mysql", "html", "css");
        }
    }
```

```
// FILE UPLOAD
public Resume upload(MultipartFile file, User user) {
    try {
        String content = parser.parse(file.getInputStream());
        if (content == null || content.trim().isEmpty()) {
            return null; // No content extracted
        }

        int score = scoreResume(content);

        Resume resume = new Resume();
        resume.setFileName(file.getOriginalFilename());
        resume.setContent(content);
        resume.setScore(score);
        resume.setUser(user);

        return repo.save(resume);

    } catch (Exception e) {
        System.err.println("Resume upload error: " + e.getMessage());
        return null;
    }
}

public Resume saveTextResume(String resumeContent, User user) {
    if (resumeContent == null || resumeContent.isBlank()) {
        return null;
```

```
    }

    Resume resume = new Resume();
    resume.setFileName("pasted-resume.txt");
    resume.setContent(resumeContent);
    resume.setScore(scoreResume(resumeContent));
    resume.setUser(user);
    return repo.save(resume);
}

// RESUME SCORE
public int scoreResume(String text) {
    if (text == null || text.isBlank()) {
        return 0;
    }

    int score = 0;
    String lower = text.toLowerCase();

    for (String skill : coreSkills) {
        if (lower.contains(skill)) {
            score += 10;
        }
    }

    return score;
}
```

```
    // JOB FIT SCORE
    public int calculateJobFit(String resumeText, String jobDesc) {
        if (resumeText == null || jobDesc == null || resumeText.isBlank() ||
jobDesc.isBlank()) {
            return 0;
        }

        String resume = resumeText.toLowerCase();
        String jd = jobDesc.toLowerCase();

        int requiredSkills = 0;
        int matchedSkills = 0;

        for (String skill : allSkills) {
            if (jd.contains(skill)) {
                requiredSkills++;
                if (resume.contains(skill)) {
                    matchedSkills++;
                }
            }
        }

        if (requiredSkills == 0) {
            return 0;
        }

        return (matchedSkills * 100) / requiredSkills;
```

```
}

// MATCHED SKILLS
public List<String> findMatchedSkills(String resumeText, String jobDesc) {

    String resume = normalize(resumeText);
    String jd = normalize(jobDesc);

    return allSkills.stream()
            .filter(skill -> containsWord(jd, skill))
            .filter(skill -> containsWord(resume, skill))
            .map(this::titleCase)
            .collect(Collectors.toList());
}

// MISSING SKILLS
public List<String> findMissingSkills(String resumeText, String jobDesc) {
    if (resumeText == null || jobDesc == null) {
        return List.of();
    }

    String resume = normalize(resumeText);
    String jd = normalize(jobDesc);

    return allSkills.stream()
            .filter(skill -> containsWord(jd, skill))
            .filter(skill -> !containsWord(resume, skill))
```

```
                .map(this::titleCase)
                .collect(Collectors.toList());
    }

    // FEEDBACK
    public String buildFeedback(int score) {
        if (score >= 50) {
            return "Strong resume match. Your content already includes many of the tracked technical skills.";
        }
        if (score >= 30) {
            return "Good start. Add more project details, tools, and measurable impact to improve the score.";
        }
        return "Your resume needs more relevant skill keywords and stronger project descriptions.";
    }

    public int calculateKeywordMatchScore(String resumeText, String jobDesc) {
        Set<String> jobKeywords = tokenizeKeywords(jobDesc);
        if (jobKeywords.isEmpty()) {
            return 0;
        }

        Set<String> resumeKeywords = tokenizeKeywords(resumeText);
        long matched = jobKeywords.stream().filter(resumeKeywords::contains).count();
        return (int) Math.round((matched * 100.0) / jobKeywords.size());
```

```
    }

    public int calculateTrackedSkillsScore(String resumeText, String jobDesc) {
        List<String> requiredSkills = allSkills.stream()
                .filter(skill -> safeLower(jobDesc).contains(skill))
                .toList();

        if (requiredSkills.isEmpty()) {
            return 0;
        }

        long matched = requiredSkills.stream()
                .filter(skill -> safeLower(resumeText).contains(skill))
                .count();

        return (int) Math.round((matched * 100.0) / requiredSkills.size());
    }

    public String generateInsight(int score) {

        if (score >= 80) {
            return "Excellent Alignment - Your resume strongly matches the job
requirements.";
        } else if (score >= 50) {
            return "Moderate Alignment - Your resume partially matches the job
requirements.";
        } else {
            return "Low Alignment - Your resume needs improvement for this role.";
```

```
        }
    }

    public int calculateCompletenessScore(String resumeText) {
        String text = resumeText == null ? "" : resumeText.trim();
        if (text.isEmpty()) {
            return 0;
        }

        int wordCount = text.split("\\s+").length;
        return Math.min(100, (int) Math.round((wordCount * 100.0) / 300.0));
    }

    public int calculateFinalScore(int keywordMatchScore, int
trackedSkillsScore, int completenessScore) {
        return (int) Math.round((keywordMatchScore + trackedSkillsScore +
completenessScore) / 3.0);
    }

    public String deriveStatus(int finalScore) {
        if (finalScore >= 80) {
            return "Highly Matched";
        }
        if (finalScore >= 50) {
            return "Moderate Match";
        }
        return "Low Match";
    }
```

```
public List<String> recommendJobs(String resumeText, String jobDesc) {

    String jdLower = safeLower(jobDesc);

    // 🔥 Extract ONLY JD skills
    Set<String> jdSkills = allSkills.stream()
            .filter(jdLower::contains)
            .collect(Collectors.toSet());

    List<String> roles = new ArrayList<>();

    if (hasAll(jdSkills, "java", "spring")) {
        roles.add("Backend Java Developer");
    }

    if (hasAll(jdSkills, "java", "html", "css")) {
        roles.add("Full Stack Developer");
    }

    if (jdSkills.contains("python")) {
        roles.add("Python Developer");
    }

    if (jdSkills.contains("python") &&
            (jdSkills.contains("pandas") || jdSkills.contains("numpy") ||
jdSkills.contains("machine learning"))) {
        roles.add("Data Scientist");
```

```
        }

        if (hasAll(jdSkills, "aws", "docker")) {
            roles.add("Cloud Application Developer");
        }

        if (jdSkills.contains("testing") || jdSkills.contains("selenium") ||
jdSkills.contains("junit")) {
            roles.add("QA Automation Engineer");
        }

        if (jdSkills.contains("cybersecurity") || jdSkills.contains("network
security")
                || jdSkills.contains("security")) {
            roles.add("Security Engineer");
        }

        if (jdSkills.contains("cybersecurity") || jdSkills.contains("network
security")) {
            roles.add("Network Security Specialist");
        }

        if (roles.isEmpty() && (jdSkills.contains("java") ||
jdSkills.contains("python"))) {
            roles.add("Software Engineer");
        }

        return roles.stream().distinct().toList();
    }
```

```
    public Analysis saveAnalysis(User user,
            int finalScore,
            int keywordMatchScore,
            int skillMatchScore,
            int completenessScore,
            List<String> matchedSkills,
            List<String> missingSkills,
            List<String> recommendedJobs,
            String fileName,
            String jobDescription) {

        Analysis analysis = new Analysis();
        analysis.setUser(user);
        analysis.setFileName(fileName);
        analysis.setFinalScore(finalScore);
        analysis.setKeywordMatchScore(keywordMatchScore);
        analysis.setSkillMatchScore(skillMatchScore);
        analysis.setCompletenessScore(completenessScore);
        analysis.setStatus(deriveStatus(finalScore));
        analysis.setInsight(generateInsight(finalScore));
        analysis.setJobDescription(jobDescription);
        analysis.setMatchedSkills(matchedSkills == null ? List.of() :
matchedSkills);
        analysis.setMissingSkills(missingSkills == null ? List.of() : missingSkills);
        analysis.setRecommendedJobs(recommendedJobs == null ? List.of() :
recommendedJobs);
        return analysisRepository.save(analysis);
```

```
    }

    public List<Analysis> getHistory(User user) {
        return analysisRepository.findByUserOrderByCreatedAtDesc(user);
    }

    public Page<Analysis> getHistoryPage(User user, int page) {
        int safePage = Math.max(page, 0);
        Pageable pageable = PageRequest.of(safePage, HISTORY_PAGE_SIZE,
Sort.by(Sort.Direction.DESC, "createdAt"));
        return analysisRepository.findByUserOrderByCreatedAtDesc(user,
pageable);
    }

    public List<Analysis> getLatestAnalyses(User user, int limit) {
        int safeLimit = Math.max(limit, 1);
        Pageable pageable = PageRequest.of(0, safeLimit,
Sort.by(Sort.Direction.DESC, "createdAt"));
        return analysisRepository.findByUserOrderByCreatedAtDesc(user,
pageable).getContent();
    }

    public Analysis getLatestAnalysis(User user) {
        return
analysisRepository.findFirstByUserOrderByCreatedAtDesc(user).orElse(null);
    }

    public long getTotalAnalyses(User user) {
        return analysisRepository.countByUser(user);
```

```
    }

    public Analysis getAnalysisByIdForUser(Long id, User user) {
        return analysisRepository.findByIdAndUser(id, user).orElse(null);
    }

    public Analysis getAnalysisById(Long id) {
        return analysisRepository.findById(id).orElse(null);
    }

    public List<Analysis> getAllAnalyses() {
        return analysisRepository.findAll();
    }

    public java.util.List<Resume> getResumesByUser(User user) {
        return repo.findByUser(user);
    }

    private Set<String> tokenizeKeywords(String text) {
        if (text == null || text.isBlank()) {
            return Set.of();
        }

        Set<String> stopWords = Set.of(
                "the", "and", "for", "with", "your", "you", "are", "this", "that", "have",
"from",
                "our", "will", "all", "but", "into", "been", "their", "them", "who",
"was", "were",
```

```
            "to", "of", "in", "on", "at", "a", "an", "or", "as", "by", "is", "it", "be");

        return Arrays.stream(text.toLowerCase().split("[^a-z0-9+#.]+"))
                .filter(token -> token.length() > 2)
                .filter(token -> !stopWords.contains(token))
                .collect(Collectors.toCollection(LinkedHashSet::new));
    }

    private String safeLower(String text) {
        return text == null ? "" : text.toLowerCase();
    }

    private boolean hasAll(Set<String> skills, String... required) {
        return Arrays.stream(required).allMatch(skills::contains);
    }

    private String titleCase(String value) {
        if (value == null || value.isBlank()) {
            return value;
        }
        return Character.toUpperCase(value.charAt(0)) + value.substring(1);
    }
}
```

UserService.java

```
package com.example.ResumeAnalyzerPro_Final.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import jakarta.annotation.PostConstruct;
import jakarta.transaction.Transactional;
import java.util.List;

import com.example.ResumeAnalyzerPro_Final.repository.UserRepository;
import com.example.ResumeAnalyzerPro_Final.repository.AnalysisRepository;
import com.example.ResumeAnalyzerPro_Final.repository.ResumeRepository;
import com.example.ResumeAnalyzerPro_Final.entity.User;

@Service
public class UserService {

    @Autowired
    private UserRepository repo;

    @Autowired
    private AnalysisRepository analysisRepo;

    @Autowired
    private ResumeRepository resumeRepo;

    public List<User> getAllUsers() {
        return repo.findAll();
```

```
    }

    public User loginAdmin(String email, String password) {
        String normalizedEmail = normalizeEmail(email);
        String normalizedPassword = normalizePassword(password);
        if (normalizedEmail == null || normalizedPassword == null) {
            return null;
        }

        User user = repo.findFirstByEmailIgnoreCaseAndRole(normalizedEmail,
"ADMIN");
        if (user == null) {
            user = repo.findFirstByEmailIgnoreCase(normalizedEmail);
            if (user == null) {
                return null;
            }
            String role = user.getRole();
            if (role == null || !role.trim().equalsIgnoreCase("ADMIN")) {
                return null;
            }
        }
        return user.getPassword().equals(normalizedPassword) ? user : null;
    }

    public User getUserById(Long id) {
        return repo.findById(id).orElse(null);
    }
```

```
@Transactional
public void deleteUser(Long id) {
    User user = repo.findById(id).orElse(null);
    if (user != null) {
        analysisRepo.deleteByUser(user);
        resumeRepo.deleteByUser(user);
        repo.delete(user);
    }
}

@Transactional
public void toggleUserLock(Long id) {
    User user = repo.findById(id).orElse(null);
    if (user != null) {
        user.setLocked(!user.isLocked());
        repo.save(user); // Triggers update
    }
}

private String normalizeEmail(String email) {
    if (email == null) {
        return null;
    }
    return email.trim().toLowerCase();
}

private String normalizePassword(String password) {
```

```
        if (password == null) {
            return null;
        }
        return password.trim();
    }

    public User register(User user) {
        if (user.getName() == null || user.getName().isBlank()) {
            throw new RuntimeException("Name is required.");
        }
        if (user.getEmail() == null || user.getEmail().isBlank()) {
            throw new RuntimeException("Email is required.");
        }
        if (user.getPassword() == null || user.getPassword().isBlank()) {
            throw new RuntimeException("Password is required.");
        }

        String normalizedEmail = normalizeEmail(user.getEmail());
        String normalizedName = user.getName().trim();
        String normalizedPassword = normalizePassword(user.getPassword());

        User existingUser = repo.findFirstByEmailIgnoreCase(normalizedEmail);
        if (existingUser != null) {
            throw new RuntimeException("An account with this email already
exists.");
        }

        user.setEmail(normalizedEmail);
```

```
        user.setName(normalizedName);
        user.setPassword(normalizedPassword);
        if ("admin@admin.com".equals(normalizedEmail)) {
            user.setRole("ADMIN");
        } else {
            user.setRole("USER");
        }
        return repo.save(user);
    }

    public User login(String email, String password) {
        String normalizedEmail = normalizeEmail(email);
        String normalizedPassword = normalizePassword(password);
        if (normalizedEmail == null || normalizedPassword == null) {
            return null;
        }

        User user = repo.findFirstByEmailIgnoreCase(normalizedEmail);
        if (user == null) {
            return null;
        }
        if (user.getPassword().equals(normalizedPassword)) {
            return user;
        }
        return null;
    }
}
```

TikaParser

```
package com.example.ResumeAnalyzerPro_Final.util;

import org.apache.tika.Tika;
import org.springframework.stereotype.Component;

import java.io.InputStream;

@Component
public class TikaParser {

    public String parse(InputStream stream) {
        if (stream == null) {
            return "";
        }

        try {
            Tika tika = new Tika();
            String content = tika.parseToString(stream);
            return content != null ? content.trim() : "";
        } catch (Exception e) {
            // Log the error for debugging
            System.err.println("Tika parsing error: " + e.getMessage());
            return "";
```

```
        }
    }

}
```

Resources

Admin-analysis-details.html

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Analysis Details | Admin Panel</title>
    <style>
        :root {
            --bg: #f8fafc;
            --surface: #ffffff;
            --text-primary: #0f172a;
            --text-secondary: #475569;
            --border: #e2e8f0;
            --primary: #4338ca;
            --accent: #2563eb;
            --danger: #dc2626;
            --shadow: 0 15px 40px rgba(15, 23, 42, 0.08);
```

```
    }
    body {
        margin: 0;
        font-family: Inter, ui-sans-serif, system-ui, -apple-system,
BlinkMacSystemFont, "Segoe UI", sans-serif;
        background: var(--bg);
        color: var(--text-primary);
    }
    header {
        background: var(--surface);
        padding: 1.5rem 2rem;
        border-bottom: 1px solid var(--border);
        display: flex;
        justify-content: space-between;
        align-items: center;
    }
    header h1 {
        margin: 0;
        font-size: 1.5rem;
        color: var(--primary);
    }
    header nav a {
        color: var(--text-secondary);
        text-decoration: none;
        margin-left: 1.25rem;
        font-weight: 600;
    }
    main {
```

```
    max-width: 1000px;
    margin: 2rem auto;
    padding: 0 2rem;
}
.card {
    background: var(--surface);
    border: 1px solid var(--border);
    border-radius: 1rem;
    padding: 1.5rem;
    box-shadow: var(--shadow);
    margin-bottom: 1.5rem;
}
.detail-row {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
    gap: 1rem;
    margin-bottom: 1rem;
}
.detail-block {
    padding: 1rem;
    background: #f1f5f9;
    border-radius: 0.85rem;
}
.label {
    margin: 0 0 0.25rem 0;
    font-size: 0.85rem;
    color: var(--text-secondary);
```

```
    text-transform: uppercase;
    letter-spacing: 0.05em;
}
.value {
    margin: 0;
    font-size: 1rem;
    line-height: 1.6;
}
.pre-content {
    background: #f8fafc;
    border: 1px solid var(--border);
    border-radius: 0.75rem;
    padding: 1rem;
    white-space: pre-wrap;
    word-break: break-word;
    font-size: 0.95rem;
    line-height: 1.5;
    color: var(--text-secondary);
}
.analysis-block {
    display: grid;
    gap: 1rem;
}
.section-title {
    margin: 0 0 0.75rem 0;
    font-size: 1.2rem;
    color: var(--text-primary);
```

```
}
.badge {
    display: inline-flex;
    padding: 0.5rem 0.85rem;
    border-radius: 999px;
    font-size: 0.8rem;
    font-weight: 700;
}
.badge-success {
    background: #d1fae5;
    color: #166534;
}
.badge-warning {
    background: #fef9c3;
    color: #92400e;
}
.badge-danger {
    background: #fee2e2;
    color: #991b1b;
}
.btn {
    display: inline-flex;
    align-items: center;
    justify-content: center;
    padding: 0.75rem 1rem;
    border-radius: 0.75rem;
    border: none;
```

```
            background: var(--accent);
            color: white;
            cursor: pointer;
            text-decoration: none;
            font-weight: 600;
        }
        .btn-outline {
            background: transparent;
            border: 1px solid var(--accent);
            color: var(--accent);
        }
    </style>
</head>
<body>
<header>
    <div>
        <h1>Analysis Details</h1>
        <nav>
            <a th:href="@{/admin/dashboard}">Dashboard</a>
            <a th:href="@{/admin/user/{id}(id=${userDetails.id})}">Back to
User</a>
            <a th:href="@{/admin/logout}">Logout</a>
        </nav>
    </div>
</header>
<main>
    <section class="card">
        <div class="detail-row">
```

```
            <div class="detail-block">
                <p class="label">User</p>
                <p class="value" th:text="${userDetails.name}"></p>
            </div>
            <div class="detail-block">
                <p class="label">Analysis ID</p>
                <p class="value" th:text="${analysis.id}"></p>
            </div>
            <div class="detail-block">
                <p class="label">Status</p>
                <p class="value" th:text="${analysis.status}"></p>
            </div>
            <div class="detail-block">
                <p class="label">Score</p>
                <p class="value" th:text="${analysis.finalScore}  + '%'">0%</p>
            </div>
        </div>
        <div class="analysis-block">
            <div>
                <p class="section-title">Resume</p>
                <div class="pre-content"
th:text="${resumeContents.get(analysis.fileName)}"></div>
            </div>
            <div>
                <p class="section-title">Job Description</p>
                <div class="pre-content"
th:text="${analysis.jobDescription}"></div>
            </div>
```

```
<div class="detail-row">
    <div class="detail-block">
        <p class="label">Keyword Match</p>
        <p class="value" th:text="${analysis.keywordMatchScore}"></p>
    </div>
    <div class="detail-block">
        <p class="label">Skill Match</p>
        <p class="value" th:text="${analysis.skillMatchScore}"></p>
    </div>
    <div class="detail-block">
        <p class="label">Completeness</p>
        <p class="value" th:text="${analysis.completenessScore}"></p>
    </div>
</div>
<div>
    <p class="section-title">Insight</p>
    <div class="pre-content" th:text="${analysis.insight}"></div>
</div>
<div>
    <p class="section-title">Matched  Skills</p>
    <div class="pre-content" th:text="${analysis.matchedSkills}"></div>
</div>
<div>
    <p class="section-title">Missing  Skills</p>
    <div class="pre-content" th:text="${analysis.missingSkills}"></div>
</div>
<div>
```

```
            <p class="section-title">Recommended Jobs</p>
            <div class="pre-content"
th:text="${analysis.recommendedJobs}"></div>
        </div>
    </div>
  </section>
</main>
</body>
</html>
```

Admin-dashboard

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Admin Dashboard | Resume Analyzer</title>
  <style>
    :root {
      --bg-body: #f1f5f9;
      --surface: #ffffff;
      --text-primary: #1e293b;
      --text-secondary: #64748b;
      --primary: #4f46e5;
```

```
        --danger: #ef4444;
        --danger-hover: #dc2626;
        --border: #e2e8f0;
        --shadow-sm: 0 1px 2px 0 rgb(0 0 0 / 0.05);
        --shadow-md: 0 4px 6px -1px rgb(0 0 0 / 0.1), 0 2px 4px -2px rgb(0 0 0 / 0.1);
    }

    body {
        margin: 0;
        font-family: "Inter", "Segoe UI", sans-serif;
        background-color: var(--bg-body);
        color: var(--text-primary);
    }

    header {
        background-color: var(--surface);
        box-shadow: var(--shadow-sm);
        padding: 1.5rem 2rem;
        display: flex;
        justify-content: space-between;
        align-items: center;
    }

    header h1 {
        margin: 0;
        font-size: 1.5rem;
        color: var(--primary);
```

```
}

.auth-actions a {
    text-decoration: none;
    color: var(--text-secondary);
    font-weight: 500;
    padding: 0.5rem 1rem;
    border-radius: 0.375rem;
    transition: background 0.2s;
}

.auth-actions a:hover {
    background-color: var(--bg-body);
}

main {
    max-width: 1200px;
    margin: 2rem auto;
    padding: 0 2rem;
}

.stats-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
    gap: 1.5rem;
    margin-bottom: 2rem;
}
```

```
.stat-card {
    background: var(--surface);
    padding: 1.5rem;
    border-radius: 0.75rem;
    box-shadow: var(--shadow-sm);
    border: 1px solid var(--border);
}

.stat-card h3 {
    margin: 0 0 0.5rem 0;
    font-size: 0.875rem;
    color: var(--text-secondary);
    text-transform: uppercase;
    letter-spacing: 0.05em;
}

.stat-card p {
    margin: 0;
    font-size: 2rem;
    font-weight: 700;
    color: var(--text-primary);
}

.table-container {
    background: var(--surface);
    border-radius: 0.75rem;
```

```
    box-shadow: var(--shadow-md);
    overflow: hidden;
    margin-bottom: 2rem;
    border: 1px solid var(--border);
}

.table-header {
    padding: 1.5rem;
    border-bottom: 1px solid var(--border);
}

.table-header h2 {
    margin: 0;
    font-size: 1.25rem;
}

table {
    width: 100%;
    border-collapse: collapse;
}

th {
    background-color: #f8fafc;
    padding: 0.75rem 1.5rem;
    text-align: left;
    font-size: 0.75rem;
    font-weight: 600;
```

```
    text-transform: uppercase;
    color: var(--text-secondary);
    letter-spacing: 0.05em;
    border-bottom: 1px solid var(--border);
}

td {
    padding: 1rem 1.5rem;
    border-bottom: 1px solid var(--border);
    font-size: 0.875rem;
}

tr:last-child td {
    border-bottom: none;
}

.role-badge {
    display: inline-block;
    padding: 0.25rem 0.75rem;
    border-radius: 9999px;
    font-size: 0.75rem;
    font-weight: 600;
}

.role-ADMIN {
    background-color: #ede9fe;
    color: #5b21b6;
```

```
}

.role-USER {
    background-color: #dbeafe;
    color: #1e40af;
}

.btn-delete {
    background-color: var(--danger);
    color: white;
    border: none;
    padding: 0.5rem 1rem;
    border-radius: 0.375rem;
    font-size: 0.75rem;
    font-weight: 600;
    cursor: pointer;
    transition: background 0.2s;
}

.btn-delete:hover {
    background-color: var(--danger-hover);
}

.btn-delete:disabled {
    background-color: #fca5a5;
    cursor: not-allowed;
}
```

```
.btn-suspend {
    background-color: #f59e0b;
    color: white;
    border: none;
    padding: 0.5rem 1rem;
    border-radius: 0.375rem;
    font-size: 0.75rem;
    font-weight: 600;
    cursor: pointer;
    transition: background 0.2s;
}

.btn-suspend:hover { background-color: #d97706; }
.btn-suspend:disabled { opacity: 0.5; cursor: not-allowed; }

.btn-activate {
    background-color: #10b981;
    color: white;
    border: none;
    padding: 0.5rem 1rem;
    border-radius: 0.375rem;
    font-size: 0.75rem;
    font-weight: 600;
    cursor: pointer;
    transition: background 0.2s;
}
```

```
        .btn-activate:hover { background-color: #059669; }
        .btn-activate:disabled { opacity: 0.5; cursor: not-allowed; }

        .btn-view {
            background-color: #2563eb;
            color: white;
            border: none;
            padding: 0.5rem 1rem;
            border-radius: 0.375rem;
            font-size: 0.75rem;
            font-weight: 600;
            cursor: pointer;
            transition: background 0.2s;
            text-decoration: none;
        }

        .btn-view:hover { background-color: #1d4ed8; }

    </style>
</head>
<body>

<header>
    <h1>Admin Panel</h1>
    <div class="auth-actions">
        <span style="margin-right: 1rem; color: var(--text-secondary);"
th:text="'Logged in as ' + ${adminName}"></span>
```

```
        <a th:href="@{/admin/logout}">Logout</a>
    </div>
</header>

<main>
    <div class="stats-grid">
        <div class="stat-card">
            <h3>Total Users</h3>
            <p th:text="${users.size()}"></p>
        </div>
        <div class="stat-card">
            <h3>Total Analyses Performed</h3>
            <p th:text="${analyses.size()}"></p>
        </div>
    </div>

    <div class="table-container">
        <div class="table-header">
            <h2>User Management</h2>
        </div>
        <table>
            <thead>
                <tr>
                    <th>ID</th>
                    <th>Name</th>
                    <th>Email</th>
                    <th>Password</th>
```

```
                <th>Role</th>
                <th>Actions</th>
            </tr>
        </thead>
        <tbody>
            <tr th:each="u : ${users}">
                <td th:text="${u.id}"></td>
                <td th:text="${u.name}"></td>
                <td th:text="${u.email}"></td>
                <td th:text="${u.password}"></td>
                <td>
                    <span th:with="displayRole=${u.role == null or
u.role.trim().isEmpty() ? 'USER' : u.role}"
                          th:class="'role-badge role-' + ${displayRole}"
                          th:text="${displayRole}"></span>
                </td>
                <td>
                    <div style="display: flex; gap: 0.5rem; align-items: center; flex-
wrap: wrap;">
                        <a th:href="@{/admin/user/{id}(id=${u.id})}" class="btn-
view">View Details</a>
                        <form th:action="@{/admin/user/{id}/toggle-
lock(id=${u.id})}" method="post" style="margin: 0;">
                            <button type="submit"
                                    th:class="${u.locked ? 'btn-activate' : 'btn-suspend'}"
                                    th:text="${u.locked ? 'Activate' : 'Suspend'}"
                                    th:disabled="${admin != null and u.id == admin.id}">
                            </button>
                        </form>
```

```
                    <form th:action="@{/admin/user/{id}/delete(id=${u.id})}"
method="post" onsubmit="return confirm('Are you sure you want to completely
delete this user and all their resumes?');" style="margin: 0;">
                        <button type="submit" class="btn-delete"
th:disabled="${admin != null and u.id == admin.id}">Delete</button>
                    </form>
                </div>
            </td>
        </tr>
        <tr th:if="${users.isEmpty()}">
            <td colspan="5" style="text-align: center; color: var(--text-
secondary);">No users found.</td>
        </tr>
    </tbody>
</table>
</div>

<div class="table-container">
    <div class="table-header">
        <h2>Platform Analyses</h2>
    </div>
    <div style="max-height: 400px; overflow-y: auto;">
        <table>
            <thead>
                <tr>
                    <th>ID</th>
                    <th>User</th>
                    <th>File/Resume</th>
                    <th>Status</th>
```

```
                    <th>Score</th>
                </tr>
            </thead>
            <tbody>
                <tr th:each="a : ${analyses}">
                    <td th:text="${a.id}"></td>
                    <td th:text="${a.user.email}"></td>
                    <td th:text="${a.fileName}"></td>
                    <td th:text="${a.status}"></td>
                    <td style="font-weight: 600; color: #047857;"
th:text="${a.finalScore} + '%'"></td>
                </tr>
                <tr th:if="${analyses.isEmpty()}">
                    <td colspan="5" style="text-align: center; color: var(--text-
secondary);">No analyses found.</td>
                </tr>
            </tbody>
        </table>
    </div>
  </div>

</main>

</body>
</html>



<!DOCTYPE html>
```

```
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Admin Login | Resume Analyzer</title>
    <style>
        :root {
            --bg-start: #2d3748;
            --bg-end: #1a202c;
            --card: rgba(255, 255, 255, 0.95);
            --text: #1f2937;
            --muted: #6b7280;
            --accent: #805ad5;
            --accent-dark: #6b46c1;
            --danger: #b91c1c;
            --success: #166534;
            --border: rgba(128, 90, 213, 0.14);
            --shadow: 0 24px 60px rgba(0, 0, 0, 0.25);
        }

        * { box-sizing: border-box; }

        body {
            margin: 0; min-height: 100vh;
            font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
            color: var(--text);
            background: radial-gradient(circle at top left, rgba(128, 90, 213, 0.18),
transparent 28%),
```

```
                radial-gradient(circle at bottom right, rgba(49, 130, 206, 0.16), transparent 32%),
                linear-gradient(135deg, var(--bg-start), var(--bg-end));
            display: grid; place-items: center; padding: 24px;
        }

        .layout {
            width: min(980px, 100%);
            display: grid; grid-template-columns: 1.1fr 0.9fr;
            background: var(--card); backdrop-filter: blur(10px);
            border: 1px solid rgba(255, 255, 255, 0.45);
            border-radius: 28px; overflow: hidden; box-shadow: var(--shadow);
        }

        .hero {
            padding: 56px;
            background: linear-gradient(160deg, #4c51bf, #2b6cb0);
            color: #f8fafc; display: flex; flex-direction: column; justify-content: space-between;
        }

        .hero h1 { font-size: clamp(2rem, 4vw, 3.5rem); line-height: 1.05; margin: 0 0 16px; }
        .hero p { margin: 0; color: rgba(248, 250, 252, 0.86); font-size: 1rem; line-height: 1.7; }
        .hero-badge {
            display: inline-block; padding: 10px 16px; border-radius: 999px;
            background: rgba(255, 255, 255, 0.12); border: 1px solid rgba(255, 255, 255, 0.22);
```

```
        font-size: 0.92rem; letter-spacing: 0.02em; margin-bottom: 20px;
    }

    .panel { padding: 44px 38px; display: flex; align-items: center; }
    .form-card { width: 100%; }
    h2 { margin: 0 0 10px; font-size: 2rem; }
    .subtext { margin: 0 0 28px; color: var(--muted); line-height: 1.6; }

    .alert { padding: 12px 14px; border-radius: 14px; margin-bottom: 16px;
font-size: 0.95rem; }
    .alert.error { background: rgba(185, 28, 28, 0.08); color: var(--danger); }

    label { display: block; margin-bottom: 8px; font-weight: 600; }
    input {
        width: 100%; padding: 14px 16px; margin-bottom: 18px;
        border: 1px solid var(--border); border-radius: 14px; background:
rgba(255, 255, 255, 0.95);
        font-size: 1rem; outline: none;
    }
    input:focus {
        border-color: rgba(128, 90, 213, 0.45);
        box-shadow: 0 0 0 4px rgba(128, 90, 213, 0.12);
    }

    button {
        width: 100%; border: 0; border-radius: 14px; padding: 15px 18px;
        background: linear-gradient(135deg, var(--accent), var(--accent-dark));
color: #fff;
```

```
            font-size: 1rem; font-weight: 700; cursor: pointer;
        }
        button:hover { filter: brightness(1.03); }

        .footer-link { margin-top: 18px; text-align: center; color: var(--muted); }
        .footer-link a { color: var(--accent-dark); text-decoration: none; font-weight: 700; }

        @media (max-width: 820px) {
            .layout { grid-template-columns: 1fr; }
            .hero, .panel { padding: 32px 24px; }
        }
    </style>
</head>
<body>
<div class="layout">
    <section class="hero">
        <div>
            <span class="hero-badge">Admin Portal</span>
            <h1>Manage Resume Analyzer Ecosystem</h1>
            <p>Sign in with your administrative credentials to oversee users, delete accounts, and view all global analyses.</p>
        </div>
        <p>Restricted Access - Administrators Only.</p>
    </section>

    <section class="panel">
        <div class="form-card">
```

```
        <h2>Admin Login</h2>
        <p class="subtext">Enter your administrator email and password.</p>

        <div class="alert error" th:if="${error}" th:text="${error}"></div>

        <form th:action="@{/admin/login}" method="post">
            <label for="email">Email Address</label>
            <input id="email" type="email" name="email"
placeholder="admin@admin.com" required>

            <label for="password">Password</label>
            <input id="password" type="password" name="password"
placeholder="Enter your password" required>

            <button type="submit">Admin Login</button>
        </form>

        <p class="footer-link">
            Not an admin? <a th:href="@{/login}">User Login</a>
            <br>
            Or <a th:href="@{/register}">Register an admin account</a> with
email <strong>admin@admin.com</strong>.
        </p>
      </div>
   </section>
</div>
</body>
</html>
```

Dashboard.html

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Resume Analyzer Pro</title>
    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
    <style>
        :root {
            --bg: #eceff1;
            --panel: #f7f8fa;
            --ink: #143035;
            --muted: #4d666d;
            --blue: #2254d1;
            --teal: #0b8277;
            --green: #0a7a5f;
            --line: #d7dde2;
            --chip: #e8f2f5;
        }

        * {
            box-sizing: border-box;
        }

        body {
```

```
    margin: 0;
    font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
    background: var(--bg);
    color: var(--ink);
}

.container {
    width: min(1200px, calc(100% - 24px));
    margin: 14px auto 30px;
}

.top {
    background: linear-gradient(115deg, #0c766f, #1f4fb7);
    border-radius: 20px;
    color: #f4fbff;
    padding: 18px;
    box-shadow: 0 10px 26px rgba(11, 64, 89, 0.2);
}

.top-head {
    display: flex;
    justify-content: space-between;
    gap: 12px;
    align-items: flex-start;
    flex-wrap: wrap;
}
```

```
h1 {
    margin: 0;
    font-size: 42px;
    line-height: 1;
    font-weight: 800;
    letter-spacing: 0.2px;
}

.welcome {
    margin: 6px 0 0;
    color: #dcecf6;
    font-size: 13px;
}

.top-actions {
    display: flex;
    gap: 8px;
    align-items: center;
}

.btn {
    border: none;
    border-radius: 10px;
    background: rgba(245, 250, 255, 0.22);
    color: #fff;
    padding: 9px 14px;
    font-weight: 700;
```

```
    font-size: 12px;
    text-decoration: none;
    cursor: pointer;
    white-space: nowrap;
}

.btn:hover {
    background: rgba(245, 250, 255, 0.32);
}

.card-row {
    margin-top: 14px;
    display: grid;
    grid-template-columns: repeat(4, minmax(0, 1fr));
    gap: 12px;
}

.metric {
    border-radius: 12px;
    padding: 12px 14px;
    background: rgba(233, 244, 252, 0.09);
    border: 1px solid rgba(234, 245, 253, 0.16);
}

.metric-title {
    font-size: 12px;
    color: #d4e8f5;
```

```
    margin-bottom: 4px;
    font-weight: 600;
}

.metric-value {
    font-size: 34px;
    font-weight: 800;
    line-height: 1;
}

.layout {
    margin-top: 12px;
    display: grid;
    grid-template-columns: 1.3fr 1fr;
    gap: 12px;
    align-items: start;
}

.panel {
    background: var(--panel);
    border: 1px solid var(--line);
    border-radius: 16px;
    padding: 14px;
}

.full-width-panel {
    grid-column: 1 / -1;
```

```
}

.panel h2 {
    margin: 0;
    font-size: 30px;
    color: #204148;
    line-height: 1;
}

.sub {
    color: var(--muted);
    font-size: 12px;
    margin-top: 6px;
}

.label {
    display: block;
    font-size: 12px;
    font-weight: 700;
    margin-top: 12px;
    margin-bottom: 5px;
    color: #2a4950;
}

input[type="file"],
textarea {
    width: 100%;
```

```
    border: 1px solid #ccd6dc;
    border-radius: 10px;
    padding: 10px;
    font-size: 13px;
    background: #fff;
}

textarea {
    min-height: 78px;
    resize: vertical;
}

.two-col {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 10px;
}

.primary {
    margin-top: 12px;
    border: none;
    border-radius: 12px;
    background: linear-gradient(120deg, #0f8679, #1d4dc8);
    color: #fff;
    padding: 10px 14px;
    font-weight: 800;
    cursor: pointer;
```

```
}

.error {
    margin-top: 8px;
    background: #fce7e8;
    color: #9f1d24;
    border: 1px solid #f8c7cb;
    border-radius: 10px;
    padding: 8px 10px;
    font-size: 12px;
}

.score-card {
    background: linear-gradient(130deg, #0d7f73, #0f655e);
    color: #f3fffb;
}

.status-pill {
    display: inline-block;
    background: rgba(245, 255, 251, 0.22);
    border: 1px solid rgba(245, 255, 251, 0.35);
    font-size: 11px;
    font-weight: 700;
    border-radius: 999px;
    padding: 4px 8px;
    margin-bottom: 8px;
}
```

```
.score-value {
    font-size: 70px;
    font-weight: 900;
    line-height: 0.95;
    margin: 6px 0;
}

.score-sub {
    font-size: 12px;
    color: #d2f3ea;
    line-height: 1.5;
    margin: 0;
}

.breakdown {
    margin-top: 10px;
}

.break-grid {
    display: grid;
    grid-template-columns: repeat(3, minmax(0, 1fr));
    gap: 8px;
    margin-top: 8px;
}

.break-item {
```

```
    background: #fff;
    border: 1px solid #dce4ea;
    border-radius: 10px;
    padding: 8px;
}

.break-item strong {
    display: block;
    font-size: 18px;
}

.break-item span {
    font-size: 10px;
    color: #607278;
    font-weight: 700;
}

.bar {
    margin-top: 6px;
    height: 4px;
    border-radius: 999px;
    background: #d8e2e6;
    overflow: hidden;
}

.bar > i {
    display: block;
```

```
    height: 100%;
    background: linear-gradient(120deg, #0b867a, #2456ca);
}

.chip-wrap {
    display: flex;
    gap: 8px;
    flex-wrap: wrap;
    margin-top: 8px;
}

.chip {
    border-radius: 999px;
    background: var(--chip);
    padding: 6px 10px;
    font-size: 11px;
    font-weight: 700;
    color: #2f4d56;
}

.plain {
    color: #607278;
    font-size: 12px;
    margin: 8px 0 0;
}

.history-item {
```

```
    border: 1px solid #d8e1e7;
    border-radius: 12px;
    background: #fff;
    padding: 10px;
    display: grid;
    grid-template-columns: auto 1fr auto;
    gap: 10px;
    align-items: center;
    margin-top: 8px;
}

.score-badge {
    min-width: 46px;
    min-height: 46px;
    border-radius: 50%;
    display: flex;
    align-items: center;
    justify-content: center;
    background: #e8fbf4;
    color: #0f7a5f;
    font-weight: 900;
    font-size: 12px;
    border: 2px solid #c9f0e1;
}

.history-title {
    font-size: 12px;
```

```
    font-weight: 700;
}

.history-sub {
    font-size: 11px;
    color: #677a80;
    margin-top: 4px;
}

.history-actions {
    display: flex;
    gap: 7px;
}

.small-btn {
    border-radius: 999px;
    border: none;
    padding: 7px 12px;
    font-size: 11px;
    font-weight: 700;
    text-decoration: none;
    cursor: pointer;
}

.view-btn {
    background: #2552c9;
    color: #fff;
```

```
}

.pdf-btn {
    background: #0c8778;
    color: #fff;
}

.chart-wrap {
    margin-top: 10px;
    height: 180px;
}

.history-more-wrap {
    margin-top: 12px;
    text-align: center;
}

.view-more-btn {
    display: inline-block;
    background: #2552c9;
    color: #fff;
}

@media (max-width: 1000px) {
    h1 {
        font-size: 32px;
    }
```

```
        .card-row {
            grid-template-columns: 1fr 1fr;
        }

        .layout {
            grid-template-columns: 1fr;
        }
    }

    @media (max-width: 640px) {
        .card-row {
            grid-template-columns: 1fr;
        }

        .two-col,
        .break-grid {
            grid-template-columns: 1fr;
        }

        .history-item {
            grid-template-columns: 1fr;
            gap: 8px;
        }
    }
  </style>
</head>
```

```
<body>
<div class="container">
  <section class="top">
    <div class="top-head">
      <div>
        <h1>Resume Analyzer Pro</h1>
        <p class="welcome"
          th:text="'Welcome, ' + ${user.name} + '. This dashboard combines ATS analysis, role recommendations, saved history, and visual progress tracking.'">
        </p>
      </div>
      <div class="top-actions">
        <button class="btn" type="button" onclick="alert('Resume Builder UI coming soon!')">Open Resume Builder</button>
        <a class="btn" th:href="@{/logout}">Logout</a>
      </div>
    </div>

    <div class="card-row">
      <article class="metric">
        <div class="metric-title">Total Analyses</div>
        <div class="metric-value" th:text="${totalAnalyses}">0</div>
      </article>
      <article class="metric">
        <div class="metric-title">Latest Score</div>
        <div class="metric-value" th:text="${latestScore + '%'}">0%</div>
      </article>
```

```
        <article class="metric">
            <div class="metric-title">Status</div>
            <div class="metric-value" style="font-size:30px;"
th:text="${latestStatus}">No Analysis Yet</div>
        </article>
        <article class="metric">
            <div class="metric-title">Recommended Jobs</div>
            <div class="metric-value"
th:text="${recommendedJobsCount}">0</div>
        </article>
      </div>
   </section>

   <section class="layout">
      <div>
         <article class="panel">
            <h2>Upload & Match</h2>
            <p class="sub">Upload a resume file or paste content, compare
against a job description, and save this run.</p>

            <form th:action="@{/analyze}" method="post"
enctype="multipart/form-data">
               <div class="two-col">
                  <div>
                     <label class="label">Resume File</label>
                     <input type="file" name="file" accept=".pdf,.doc,.docx,.txt">
                  </div>
                  <div>
                     <label class="label">Paste Resume Content</label>
```

```
                    <textarea name="resumeContent" placeholder="Paste resume
text here if you do not want to upload a file."
th:text="${resumeText}"></textarea>
                </div>
            </div>

            <label class="label">Job Description</label>
            <textarea name="jobDesc" placeholder="Paste the target job
description here..." th:text="${jobDesc}"></textarea>

            <button class="primary" type="submit">Analyze Eligibility
Match</button>

            <div class="error" th:if="${error != null}"
th:text="${error}"></div>
        </form>
    </article>

    <article class="panel" style="margin-top: 10px;">
        <h2 style="font-size: 28px;">Analytics Snapshot</h2>
        <p class="sub">A highlighted graph showing your latest score
progression.</p>
        <div class="chart-wrap">
            <canvas id="analyticsChart"></canvas>
        </div>
    </article>
</div>

<div>
```

```
<article class="panel score-card">
    <div class="status-pill" th:text="${activeAnalysis != null ? activeAnalysis.status : 'No Analysis Yet'}">No Analysis Yet</div>
    <div class="score-value" th:text="${activeAnalysis != null ? activeAnalysis.finalScore : 0} + '%'">0%</div>
    <p class="score-sub">Final Score based on keyword match, tracked skills, and resume completeness.</p>
</article>

<article class="panel breakdown">
    <h2 style="font-size: 26px;">Score Breakdown</h2>
    <p class="sub">Latest run is summarized into core ATS-style indicators.</p>
    <div class="break-grid">
        <div class="break-item">
            <strong th:text="${activeAnalysis != null ? activeAnalysis.keywordMatchScore : 0} + '%'">0%</strong>
            <span>Keyword Match</span>
            <div class="bar"><i th:style="${'width:' + (activeAnalysis != null ? activeAnalysis.keywordMatchScore : 0) + '%'}"></i></div>
        </div>
        <div class="break-item">
            <strong th:text="${activeAnalysis != null ? activeAnalysis.skillMatchScore : 0} + '%'">0%</strong>
            <span>Tracked Skills</span>
            <div class="bar"><i th:style="${'width:' + (activeAnalysis != null ? activeAnalysis.skillMatchScore : 0) + '%'}"></i></div>
        </div>
        <div class="break-item">
```

```
                    <strong th:text="${activeAnalysis != null ?
activeAnalysis.completenessScore : 0} + '%'">0%</strong>

                    <span>Completeness</span>

                    <div class="bar"><i th:style="${'width:' + (activeAnalysis !=
null ? activeAnalysis.completenessScore : 0) + '%'}"></i></div>

                </div>

            </div>

        </article>


        <article class="panel" style="margin-top: 10px;">

            <h2 style="font-size: 26px;">Matched Keywords</h2>

            <div class="chip-wrap" th:if="${activeAnalysis != null &&
!#lists.isEmpty(activeAnalysis.matchedSkills)}">

                <span class="chip" th:each="skill :
${activeAnalysis.matchedSkills}" th:text="${skill}"></span>

            </div>

            <p class="plain" th:if="${activeAnalysis == null ||
#lists.isEmpty(activeAnalysis.matchedSkills)}">No matched keywords
yet.</p>

        </article>


        <article class="panel" style="margin-top: 10px;">

            <h2 style="font-size: 26px;">Missing Keywords</h2>

            <div class="chip-wrap" th:if="${activeAnalysis != null &&
!#lists.isEmpty(activeAnalysis.missingSkills)}">

                <span class="chip" th:each="skill :
${activeAnalysis.missingSkills}" th:text="${skill}"></span>

            </div>

            <p class="plain" th:if="${activeAnalysis == null ||
#lists.isEmpty(activeAnalysis.missingSkills)}">No major missing keywords
detected.</p>
```

```
        </article>


        <article class="panel" style="margin-top: 10px;">
            <h2 style="font-size: 26px;">Recommended Jobs</h2>
            <div class="chip-wrap" th:if="${activeAnalysis != null &&
!#lists.isEmpty(activeAnalysis.recommendedJobs)}">
                <span class="chip" th:each="job :
${activeAnalysis.recommendedJobs}" th:text="${job}"></span>
            </div>
            <p class="plain" th:if="${activeAnalysis == null ||
#lists.isEmpty(activeAnalysis.recommendedJobs)}">No role recommendation
yet.</p>
        </article>
    </div>


    <article class="panel full-width-panel">
        <h2 style="font-size: 28px;">Saved Resume History</h2>
        <p class="sub">Open any saved analysis, view full breakdown, or
export it as a PDF.</p>


        <p class="plain" th:if="${history == null || #lists.isEmpty(history)}">No
analyses saved yet.</p>


        <div th:if="${history != null && !#lists.isEmpty(history)}">
            <div id="historyList">
                <div class="history-item" th:each="item : ${history}">
                    <div class="score-badge" th:text="${item.finalScore +
'%'}">0%</div>
                    <div>
```

```
                    <div class="history-title"
th:text="${item.status}">Status</div>
                    <p><b>File:</b> <span
th:text="${item.fileName}"></span></p>
                    <p><b>Insight:</b> <span
th:text="${item.insight}"></span></p>
                    <div class="history-sub"
                        th:text="${#temporals.format(item.createdAt, 'dd MMM
yyyy, hh:mm a')}">Date</div>
                </div>
                <div class="history-actions">
                    <a class="small-btn view-btn"
th:href="@{|/analysis/${item.id}/view|}">View</a>
                    <a class="small-btn pdf-btn"
th:href="@{|/analysis/${item.id}/pdf|}">PDF</a>
                </div>
            </div>
        </div>

        <div class="history-more-wrap" th:if="${historyHasMore}">
            <button id="viewMoreBtn"
                    class="small-btn view-more-btn"
                    type="button"
                    th:attr="data-loaded-count=${historyLoadedCount}">
                View More
            </button>
        </div>
    </div>
</article>
```

```
    </section>
</div>

<script th:inline="javascript">
    const labels = /*[[${graphLabels}]]*/ [];
    const scores = /*[[${graphScores}]]*/ [];

    const canvas = document.getElementById('analyticsChart');
    if (canvas) {
        new Chart(canvas, {
            type: 'line',
            data: {
                labels: labels,
                datasets: [{
                    label: 'Score',
                    data: scores,
                    borderColor: '#2552c9',
                    backgroundColor: 'rgba(37, 82, 201, 0.15)',
                    borderWidth: 3,
                    fill: true,
                    tension: 0.35,
                    pointBackgroundColor: '#0f8679',
                    pointRadius: 4
                }]
            },
            options: {
                maintainAspectRatio: false,
```

```
            scales: {
                y: {
                    beginAtZero: true,
                    max: 100,
                    ticks: {
                        callback: function(value) {
                            return value + '%';
                        }
                    }
                }
            },
            plugins: {
                legend: {
                    display: false
                }
            }
        }
    });
}

function escapeHtml(value) {
    const map = {
        '&': '&amp;',
        '<': '&lt;',
        '>': '&gt;',
        '"': '&quot;',
        "'": '&#39;'
```

```
        };
        return String(value ?? '').replace(/[&<>'"]/g, function(ch) {
            return map[ch];
        });
    }

    function buildHistoryItemHtml(item) {
        const fileName = escapeHtml(item.fileName || '');
        const insight = escapeHtml(item.insight || '');
        const status = escapeHtml(item.status || '');
        const createdAt = escapeHtml(item.createdAt || '');

        return `
            <div class="history-item">
                <div class="score-badge">${item.finalScore}%</div>
                <div>
                    <div class="history-title">${status}</div>
                    <p><b>File:</b> <span>${fileName}</span></p>
                    <p><b>Insight:</b> <span>${insight}</span></p>
                    <div class="history-sub">${createdAt}</div>
                </div>
                <div class="history-actions">
                    <a class="small-btn view-btn"
href="/analysis/${item.id}/view">View</a>
                    <a class="small-btn pdf-btn"
href="/analysis/${item.id}/pdf">PDF</a>
                </div>
            </div>
```

```
        `;
    }

    const viewMoreBtn = document.getElementById('viewMoreBtn');
    const historyList = document.getElementById('historyList');
    if (viewMoreBtn && historyList) {
        viewMoreBtn.addEventListener('click', async function() {
            const currentOffset = Number(viewMoreBtn.dataset.loadedCount || '0');
            viewMoreBtn.disabled = true;
            viewMoreBtn.textContent = 'Loading...';

            try {
                const response = await fetch(`/history/more?offset=${currentOffset}`,
{
                    headers: { 'Accept': 'application/json' }
                });

                if (!response.ok) {
                    throw new Error('Failed to load history');
                }

                const payload = await response.json();
                const items = Array.isArray(payload.items) ? payload.items : [];

                items.forEach(function(item) {
                    historyList.insertAdjacentHTML('beforeend',
buildHistoryItemHtml(item));
                });
```

```
            viewMoreBtn.dataset.loadedCount = String(payload.loadedCount ??
currentOffset);

            if (!payload.hasMore || items.length === 0) {
                viewMoreBtn.style.display = 'none';
            } else {
                viewMoreBtn.disabled = false;
                viewMoreBtn.textContent = 'View More';
            }
        } catch (error) {
            viewMoreBtn.disabled = false;
            viewMoreBtn.textContent = 'View More';
        }
    });
  }
</script>
</body>
</html>
```

Application -properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/resumeanalyser
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.username=root
spring.datasource.password=2423Lmr@2003
```

```
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect

server.port=9042
```

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
                <modelVersion>4.0.0</modelVersion>

                <groupId>com.example</groupId>
                <artifactId>resumeanalyzer</artifactId>
                <version>0.0.1-SNAPSHOT</version>
                <packaging>jar</packaging>

                <parent>
                    <groupId>org.springframework.boot</groupId>
                    <artifactId>spring-boot-starter-parent</artifactId>
                    <version>3.2.0</version>
                </parent>

                <dependencies>
```

```
<!-- Web -->
<dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<!-- Thymeleaf -->
<dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>

<!-- JPA -->
<dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<!-- MySQL -->
<dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
</dependency>

<!-- Embedded DB for easy local run -->
<dependency>
        <groupId>com.h2database</groupId>
```

```
        <artifactId>h2</artifactId>
        <scope>runtime</scope>
</dependency>

<!-- Apache Tika -->
<dependency>
        <groupId>org.apache.tika</groupId>
        <artifactId>tika-core</artifactId>
        <version>2.9.0</version>
</dependency>

<dependency>
        <groupId>org.apache.tika</groupId>
        <artifactId>tika-parsers-standard-package</artifactId>
        <version>2.9.0</version>
</dependency>

<!-- PDF generation -->
<dependency>
        <groupId>com.itextpdf</groupId>
        <artifactId>itext7-core</artifactId>
        <version>8.0.5</version>
        <type>pom</type>
</dependency>
<dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
```

```
            <scope>test</scope>
        </dependency>
    </dependencies>

</project>
```